

Divide y vencerás

Divide y vencerás es una técnica para diseñar algoritmos que consiste en descomponer el caso que haya que resolver en un cierto número de subcasos más pequeños del mismo problema, resolver sucesiva e independientemente todos estos subcasos, y combinar después las soluciones obtenidas de esta manera para obtener la solución del caso original. Las dos preguntas que surgen de modo natural son: ¿por qué querría nadie hacer esto? y ¿cómo se resuelven los subcasos? La eficiencia de la técnica de divide y vencerás se debe a la respuesta a esta última pregunta.

7.1 INTRODUCCIÓN: MULTIPLICACIÓN DE ENTEROS MUY GRANDES

Consideremos una vez más el problema de multiplicar enteros muy grandes. Recordemos que el algoritmo clásico (figura 1.1) que aprendemos en la escuela requiere un tiempo en $\Theta(n^2)$ para multiplicar números de n cifras. Estamos tan acostumbrados a este algoritmo que quizá nunca haya tenido dudas acerca de su optimalidad. ¿Podemos hacerlo mejor? La multiplicación *à la russe* (figura 1.2) no ofrece mejoras del tiempo de ejecución. Otro algoritmo que discutíamos en la Sección 1.2, y que llamábamos técnica de «divide y vencerás» (figura 1.3) consistía en reducir la multiplicación de dos números de n cifras a cuatro multiplicaciones de números de $\frac{n}{2}$ cifras. Lamentablemente, el algoritmo resultante no proporciona ninguna mejora con respecto al algoritmo clásico de multiplicación, a no ser que seamos más inteligentes. Para superar al algoritmo clásico, debemos encontrar una forma de reducir la multiplicación original no a cuatro sino a *tres* multiplicaciones de números de tamaño mitad.

Ilustraremos el proceso con el ejemplo utilizado en la Sección 1.2: la multiplicación de 981 por 1.234. En primer lugar se rellena el operando más corto con un cero no significativo para hacerlo de longitud igual a la del más largo; de esta manera 981 se convierte en 0981. Entonces partimos en dos mitades ambos operandos: 0981 da lugar a $w = 09$ y $x = 81$, y 1.234 da lugar a $y = 12$ y $z = 34$. Obsérvese que $981 = 10^2w + x$ y que $1.234 = 10^2y + z$. Por tanto, el producto requerido se puede calcular en la forma siguiente:

$$\begin{aligned}
 981 \times 1.234 &= (10^2w + x) \times (10^2y + z) \\
 &= 10^4wy + 10^2(wz + xy) + xz \\
 &= 1.080.000 + 127.800 + 2.754 = 1.210.554
 \end{aligned}$$

Si piensa que no hemos hecho otra cosa que reescribir el algoritmo de la Sección 1.2 con más símbolos, estará en lo cierto. El procedimiento anterior sigue necesitando cuatro multiplicaciones de números de tamaño mitad: wy , wz , xy y xz .

La observación clave es que no hay necesidad de calcular wz y xy ; lo único que necesitamos realmente es la *suma* de estos dos términos. ¿Es posible obtener $wz + xy$ a costa de una sola multiplicación? Esto parece imposible hasta que recordamos que también se necesitan los valores de wy y de xz para aplicar la fórmula anterior. Teniendo esto en cuenta, consideremos el producto

$$r = (w + x) \times (y + z) = wy + (wz + xy) + xz$$

Al cabo de una sola multiplicación, obtenemos la suma de los tres términos necesarios para calcular el producto deseado. Esto sugiere que procedamos en la forma siguiente:

$$\begin{aligned}
 p &= wy = 09 \times 12 &&= 108 \\
 q &= xz = 81 \times 34 &&= 2.754 \\
 r &= (w + x) \times (y + z) &&= 90 \times 46 = 4.140
 \end{aligned}$$

Y finalmente

$$\begin{aligned}
 981 \times 1.234 &= 10^4p + 10^2(r - p - q) + q \\
 &= 1.080.000 + 127.800 + 2.754 = 1.210.554
 \end{aligned}$$

De esta manera, el producto de 981 por 1.234 se puede reducir a *tres* multiplicaciones de números de dos cifras (09×12 , 81×34 y 90×46) junto con un cierto número de desplazamientos (multiplicaciones por potencias de 10), adiciones y sustracciones.

Desde luego, el número de sumas —contando las restas como si fueran sumas— es mayor que en el algoritmo divide y vencerás original de la Sección 1.2. ¿Merece la pena efectuar cuatro sumas más para ahorrar una multiplicación? La respuesta es negativa cuando se multiplican números pequeños como los de nuestro ejemplo. Sin embargo, merece la pena cuando los números que hay que multiplicar son grandes, y cada vez más cuanto mayores sean los números. Cuando los operandos son grandes, el tiempo requerido para las sumas es despreciable frente al tiempo que requiere una sola multiplicación. Entonces parece razonable esperar que la reducción de cuatro multiplicaciones a tres nos permita reducir en un 25% el tiempo requerido para grandes multiplicaciones. Como veremos, nuestro ahorro va a ser significativamente mayor.

Para ayudarnos a comprender lo que hemos logrado, supongamos que una implementación dada del algoritmo clásico de multiplicación requiere un tiempo

$h(n) = cn^2$ para multiplicar dos números de n cifras, para alguna constante c que dependerá de la implementación (esto es una simplificación, porque en realidad el tiempo requerido tendría una forma más complicada, tal como $cn^2 + bn + a$). De manera similar, sea $g(n)$ el tiempo requerido por el algoritmo divide y vencerás para multiplicar dos números de n cifras, *sin* contar el tiempo necesario para efectuar las tres multiplicaciones de números de tamaño mitad. En otras palabras, $g(n)$ es el tiempo necesario para las sumas, desplazamientos y operaciones adicionales varias. Resulta sencillo implementar estas operaciones de tal manera que $g(n) \in \Theta(n)$. Ignore por el momento lo que sucede si n es impar o si los números no son de la misma longitud.

Si las tres multiplicaciones de números de tamaño mitad se efectúan mediante el algoritmo clásico, el tiempo necesario para multiplicar los dos números de n cifras es:

$$3h(n/2) + g(n) = 3c(n/2)^2 + g(n) = \frac{3}{4} cn^2 + g(n) = \frac{3}{4} h(n) + g(n)$$

Dado que $h(n) \in \Theta(n^2)$ y $g(n) \in \Theta(n)$, el término $g(n)$ resulta despreciable en comparación con $\frac{3}{4}h(n)$ cuando n es suficientemente grande, lo cual significa que hemos ganado aproximadamente un 25% de velocidad en comparación con el algoritmo clásico, tal como anticipábamos. Aun cuando esta mejora no es despreciable, no hemos conseguido modificar el orden del tiempo requerido: el nuevo algoritmo sigue requiriendo un tiempo cuadrático.

Para mejorar esto, volvemos a la cuestión planteada en el párrafo inicial: ¿cómo hay que resolver los subejemplares? Si son pequeños, es posible que el algoritmo clásico siga siendo la mejor manera de proceder. Sin embargo, cuando los subejemplares sean suficientemente grandes, ¿no sería mejor utilizar nuestro nuevo algoritmo de *forma recursiva*? ¡La idea es análoga a beneficiarse de una cuenta bancaria que componga los pagos de interés! Cuando hacemos esto, obtenemos un algoritmo que puede multiplicar dos números de n cifras en un tiempo $t(n) = 3t(n/2) + g(n)$ cuando n es par y suficientemente grande. Esta recurrencia es parecida a la que estudiábamos en la Sección 4.7.1, y al ejemplo 4.7.10; al resolverla se obtiene que $t(n) \in \Theta(n^{\lg 3} \mid n \text{ es una potencia de } 2)$. Tenemos que contentarnos con nuestra notación asintótica condicional porque todavía no hemos atacado la cuestión de cómo se multiplican números de longitud impar; véase el Problema 7.1.

Dado que $\lg 3 \approx 1,585$ es menor que 2, este algoritmo puede multiplicar dos enteros grandes mucho más deprisa que el algoritmo clásico de multiplicación, y cuanto mayor sea n , más merecerá la pena esta mejora. Es probable que una buena implementación no utilice la base 10, si no más bien la mayor de las bases para las que el *hardware* admita multiplicar directamente dos dígitos. Recuerde que el rendimiento de este algoritmo y el del algoritmo clásico se han comparado empíricamente al final de la Sección 2.7.3.

Un factor importante para la eficiencia práctica de este algoritmo de multiplicación, y de hecho de cualquier algoritmo del tipo divide y vencerás, consiste en saber cuándo hay que dejar de dividir los casos para empezar a utilizar directamen-

te el algoritmo clásico. Aun cuando el enfoque de divide y vencerás es mejor cuanto más grande es el caso, lo cierto es que puede resultar *más lento* que el algoritmo clásico en casos que sean demasiado pequeños. Por tanto, un algoritmo de divide y vencerás debe de evitar seguir avanzando recursivamente cuando el tamaño de los casos ya no lo justifique. Volveremos a este asunto en la sección siguiente.

Por sencillez, hay varios asuntos importantes que han quedado en el tintero hasta este momento. ¿Cómo se tratan los números de longitud impar? Aun cuando las dos mitades del multiplicando y del multiplicador sean de tamaño $n/2$, puede suceder que su *suma* se desborde, y sea de un tamaño una unidad mayor. Por tanto, era ligeramente incorrecto afirmar que $r = (w + x) \times (y + z)$ necesita una multiplicación de números de tamaño mitad. ¿Cómo afecta esto al análisis del tiempo de ejecución? ¿Cómo multiplicamos dos números de diferentes tamaños? ¿Existen otras operaciones aritméticas, además de la multiplicación, que podamos manejar más eficientemente que mediante el uso de los algoritmos clásicos?

Los números de longitud impar se multiplican fácilmente partiéndolos del modo más equitativo posible: un número de n cifras se parte en un número de $\lfloor n/2 \rfloor$ cifras y otro de $\lceil n/2 \rceil$ cifras. La segunda cuestión es más peliaguda. Consideremos la multiplicación de 5.678 por 6.789. Nuestro algoritmo parte los operandos en la forma $w = 56$, $x = 78$, $y = 67$ y $z = 89$. Las tres multiplicaciones de tamaño mitad implicadas son:

$$\begin{aligned} p &= wy = 56 \times 67 \\ q &= xz = 78 \times 69, y \\ r &= (w + x) \times (y + z) = 134 \times 156 \end{aligned}$$

La tercera multiplicación usa números de *tres* cifras, y por tanto no son realmente de tamaño mitad en comparación con los números originales de cuatro cifras. Sin embargo, el tamaño de $w + x$ y el de $y + z$ no puede sobrepasar $1 + \lceil n/2 \rceil$.

Para simplificar el análisis, supongamos que $t(n)$ denota el tiempo requerido por este algoritmo en el caso peor para multiplicar dos números cuyo tamaño no sea mayor que n (en lugar de ser exactamente n). Por definición, $t(n)$ es una función no decreciente. Cuando n es suficientemente grande, nuestro algoritmo reduce la multiplicación de dos números de tamaño menor o igual que n a tres multiplicaciones $p = wy$, $q = xz$ y $r = (w + x) \times (y + x)$, de números de tamaños que son menores o iguales que $\lfloor n/2 \rfloor$, $\lceil n/2 \rceil$ y $1 + \lceil n/2 \rceil$, respectivamente, además de algunas manipulaciones sencillas que requieren un tiempo en $O(n)$. Por tanto, existe una constante real y positiva c tal que

$$t(n) \leq t(\lfloor n/2 \rfloor) + t(\lceil n/2 \rceil) + t(1 + \lceil n/2 \rceil) + cn$$

para todo n suficientemente grande. Esta es precisamente la recurrencia estudiada en el ejemplo 4.7.14, y que da lugar al ya familiar $t(n) \in O(n^{\lg 3})$. Por tanto, siempre es posible multiplicar números de n cifras en un tiempo que está en $O(n^{\lg 3})$. Un análisis de este algoritmo en el caso peor muestra que de hecho $t(n) \in \Theta(n^{\lg 3})$, pero es-

te resultado tiene un interés limitado porque existen algoritmos de multiplicación todavía más rápidos; véanse los problemas 7.2 y 7.3.

Volviendo a la cuestión de multiplicar números de distintos tamaños, sean u y v dos enteros de tamaños m y n respectivamente. Si m y n no difieren en más de un factor 2, lo mejor es rellenar el operando más pequeño con ceros no significativos, para hacerlo de longitud igual a la del otro operando, tal como se hacía al multiplicar 981 por 1.234. Sin embargo, este enfoque no resulta recomendable cuando uno de los operandos es mucho mayor que el otro. ¡Podría ser incluso peor que utilizar el algoritmo clásico de multiplicación! Sin pérdida de generalidad, supongamos que $m \leq n$. El algoritmo divide y vencerás utilizado con relleno y el algoritmo clásico requieren un tiempo que está en $\Theta(n^{\lg 3})$, y en $\Theta(mn)$ respectivamente, para calcular el producto de u y v . Considerando que la constante oculta del primero tiene probabilidades de ser más grande que la del segundo, vemos que el algoritmo divide y vencerás con relleno es *más lento* que el algoritmo clásico cuando $m < n^{\lg(3/2)}$, y en particular cuando $m < \sqrt{n}$.

Sin embargo, es fácil combinar los dos algoritmos para obtener un algoritmo realmente mejor. La idea es segmentar el operando más largo, v , en bloques de tamaño m , y utilizar entonces el algoritmo divide y vencerás para multiplicar u por cada bloque de v , de tal manera que se utiliza el algoritmo divide y vencerás para multiplicar parejas de operandos de igual tamaño. El producto final de u y v se obtiene entonces fácilmente mediante adiciones y desplazamientos sencillos. El tiempo total de ejecución está dominado por la necesidad de efectuar $\lceil n / m \rceil$ multiplicaciones de números de m cifras. Dado que cada una de estas multiplicaciones más pequeñas requiere un tiempo que está en $\Theta(m^{\lg 3})$, y teniendo en cuenta que $\lceil n / m \rceil \in \Theta(n / m)$, el tiempo total de ejecución necesario para multiplicar un número de n cifras por un número de m cifras está en $\Theta(nm^{\lg(3/2)})$ cuando $m \leq n$.

La multiplicación no es la única operación interesante que utiliza enteros grandes. La exponenciación modular también es crucial para la criptografía moderna; véase la Sección 7.8. La división entera, las operaciones de módulo y el cálculo de la parte entera de una raíz cuadrada se pueden efectuar todas ellas en un tiempo cuyo orden es el mismo que se requiere para la multiplicación; véase la Sección 12.4. Otras operaciones importantes, tales como el cálculo del máximo común divisor, pueden ser inherentemente más difíciles de calcular y no se tratarán aquí.

7.2 EL CASO GENERAL

La multiplicación de enteros muy grandes no es un ejemplo aislado del beneficio que se puede obtener empleando el enfoque de divide y vencerás. Consideremos un problema arbitrario, y sea *ad hoc* un algoritmo sencillo capaz de resolver el problema. De *ad hoc* nos interesa que sea eficiente para casos pequeños, pero su rendimiento para casos grandes no nos concierne. Lo llamamos *subalgoritmo básico*. El algoritmo clásico de multiplicación es un ejemplo de subalgoritmo básico.

El caso general de los algoritmos de divide y vencerás es como sigue:

función $DV(x)$

si x es suficientemente pequeño o sencillo, **entonces proporcionar** *ad hoc*(x)
 descomponer x en casos más pequeños $x_1, x_2, \dots, x_\ell$
para $i \leftarrow 1$ **hasta** ℓ **hacer** $y_i \leftarrow DV(x_i)$
 recombinar los y_i para obtener una solución y de x
devolver y

Algunos algoritmos de divide y vencerás no siguen exactamente este esquema: por ejemplo, pueden necesitar que el primer subejemplar esté resuelto antes de formular el segundo subejemplar; véase la Sección 7.5.

El número de subejemplares, ℓ , suele ser pequeño e independiente del caso particular que haya que resolver. Cuando $\ell = 1$, no tiene mucho sentido «descomponer x en un caso más sencillo x_1 », y es difícil justificar el nombre de *divide y vencerás* que se le da a esta técnica. Sin embargo, tiene sentido reducir la resolución de un caso muy grande a la de uno más pequeño. Entonces, divide y vencerás recibe el nombre de *reducción (simplificación)*; véanse las Secciones 7.3 y 7.7. Cuando se utiliza la simplificación, a veces es posible sustituir la recursividad inherente a divide y vencerás por un bucle iterativo. Cuando se implementa en un lenguaje convencional como Pascal y en una máquina convencional que utilice un compilador poco sofisticado, un algoritmo iterativo tiene probabilidades de ser más rápido que la versión recursiva, aunque sólo por una constante multiplicativa. Por otra parte, puede ser posible ahorrar una cantidad considerable de memoria de esta manera: para un caso de tamaño n , el algoritmo recursivo utiliza una pila cuyo tamaño está en $\Omega(\lg n)$, y en casos malos incluso en $\Omega(n)$.

Para que el enfoque divide y vencerás merezca la pena, es necesario que se cumplan tres condiciones. La decisión de utilizar el subalgoritmo básico en lugar de hacer llamadas recursivas debe tomarse cuidadosamente; tiene que ser posible descomponer el ejemplar en subejemplares y recomponer las soluciones parciales de forma bastante eficiente, y los subejemplares deben ser en lo posible aproximadamente del mismo tamaño. La mayoría de los algoritmos de divide y vencerás son tales que el tamaño de los ℓ subejemplares es aproximadamente n/b para alguna constante b , en donde n es el tamaño del caso original. Por ejemplo, nuestro algoritmo de divide y vencerás para multiplicar enteros muy grandes requiere un tiempo en $\Theta(n)$ para descomponer el caso original en tres subejemplares de tamaños próximos a la mitad, y para recombinar las soluciones parciales: $\ell = 3$ y $b = 2$.

El análisis de tiempos de ejecución para estos algoritmos de divide y vencerás es casi automático, gracias a los ejemplos 4.7.13 y 4.7.16. Sea $g(n)$ el tiempo requerido por DV en casos de tamaño n , sin contar el tiempo necesario para llamadas recursivas. El tiempo total $t(n)$ requerido por este algoritmo de divide y vencerás es parecido a:

$$t(n) = \ell t(n \div b) + g(n)$$

siempre que n sea suficientemente grande. Si existe un entero k tal que $g(n) \in \Theta(n^k)$, el ejemplo 4.7.16 es aplicable para concluir que:

$$t(n) \in \begin{cases} \Theta(n^k) & \text{si } t < b^k \\ \Theta(n^k \log n) & \text{si } t = b^k \\ \Theta(n^{\log_b t}) & \text{si } t > b^k. \end{cases} \quad (7.1)$$

Las técnicas utilizadas en la Sección 4.7.6 y en el ejemplo 4.7.14 son aplicables en general para obtener la misma conclusión aun cuando algunos de los subejemplares sean de un tamaño que sea distinto de $\lfloor n/b \rfloor$ en una constante aditiva, y en particular si algunos de los subejemplares son de tamaño $\lceil n/b \rceil$. Como ejemplo, nuestro algoritmo de divide y vencerás para la multiplicación de grandes enteros se caracteriza por $\ell = 3$, $b = 2$ y $k = 1$. Dado que $\ell > b^k$, es aplicable el tercer caso y obtenemos inmediatamente que el algoritmo requiere un tiempo que está en $\Theta(n^{\lg 3})$ sin tener que preocuparnos por el hecho de que dos de los subejemplares son de tamaños $\lceil n/2 \rceil$ y $1 + \lceil n/2 \rceil$ en lugar de $\lfloor n/2 \rfloor$. En situaciones más complicadas, en las que $g(n)$ no es del orden exacto de un polinomio, puede ser aplicable el Problema 4.44.

Queda por ver la forma de determinar si hay que dividir el caso y hacer llamadas recursivas, o si el caso es tan sencillo que resulta mejor invocar directamente al subalgoritmo básico. Aun cuando esta decisión no afecta al orden del tiempo de ejecución del algoritmo, nos interesa que la constante multiplicativa oculta en la notación Θ sea lo más pequeña posible. En la mayoría de los algoritmos de divide y vencerás, esta decisión se basa en un sencillo *umbral*, que suele denotarse mediante n_0 . El subalgoritmo básico se emplea para resolver todos aquellos casos cuyo tamaño no supere n_0 .

Volvamos al problema de multiplicar enteros muy grandes, para ver por qué es importante la selección de un umbral, y la forma de seleccionarlo. Para evitar oscurecer las cuestiones esenciales, utilizaremos una fórmula de recurrencia simplificada para el tiempo de ejecución del algoritmo divide y vencerás para multiplicar enteros grandes:

$$t(n) = \begin{cases} h(n) & \text{si } n \leq n_0 \\ 3t(\lceil n/2 \rceil) + g(n) & \text{en caso contrario} \end{cases}$$

en donde $h(n) \in \Theta(n^2)$ y $g(n) \in \Theta(n)$.

Consideremos, por ejemplo, una implementación en la cual $h(n) = n^2$ y $g(n) = 16n$ microsegundos. Supongamos que tenemos que multiplicar dos números de 5.000 cifras. Si el algoritmo divide y vencerás procede de forma recursiva hasta obtener subejemplares de tamaño 1, esto es, si $n_0 = 1$, entonces se necesitan más de 41 segundos para calcular el producto. Esto es ridículo, puesto que estos dos números se pueden multiplicar en 25 segundos empleando el algoritmo clásico. El algoritmo clásico es ligeramente mejor que el algoritmo divide y vencerás incluso para multiplicar números de 32.789 cifras, caso en el que ambos algoritmos requieren más de un cuarto de

hora de tiempo de cálculo ¡para efectuar una única multiplicación! ¿Debemos concluir que el algoritmo de divide y vencerás nos permite pasar de un algoritmo cuadrático a uno cuyo tiempo de ejecución está en $\Theta(n^{\lg 3})$, pero sólo a costa de un incremento tan enorme de la constante oculta que el nuevo algoritmo nunca resulta económico en casos de tamaño razonable? Afortunadamente, no es así: para seguir con nuestro ejemplo, se pueden multiplicar números de 5.000 cifras en poco más de 6 segundos, siempre y cuando seleccionemos de forma inteligente el umbral n_0 ; en este caso, $n_0 = 64$ es una buena opción. Con el mismo umbral, se necesita poco más de dos minutos para multiplicar dos números de 32.789 cifras.

La selección del mejor umbral se ve complicada por el hecho de que el valor óptimo depende en general no sólo del algoritmo en cuestión, sino también de la implementación particular. Además, no existe en general un valor uniformemente mejor del umbral. En nuestro ejemplo, lo mejor es utilizar el algoritmo clásico para multiplicar números de 67 cifras, mientras que es mejor utilizar la recurrencia una sola vez para multiplicar números de 66 cifras. Por tanto, 67 es mejor que 64 como umbral en el primer caso, mientras que en el segundo es mejor lo contrario. En lo sucesivo, emplearemos el término «umbral óptimo» para indicar *casi óptimo*.

¿Y entonces, qué seleccionamos como umbral? Dada una implementación particular, el umbral óptimo se puede determinar empíricamente. Variamos el valor del umbral y el tamaño de los casos usados para nuestra prueba, y cronometramos la implementación en cierto números de casos. Suele ser posible estimar un umbral óptimo tabulando los resultados de estas pruebas, o incluso dibujando unos cuantos diagramas. Sin embargo, los cambios del umbral dentro de un cierto intervalo pueden no tener efecto sobre la eficiencia del algoritmo cuando sólo se consideren casos de un tamaño específico. Por ejemplo, se necesita exactamente el mismo tiempo para multiplicar dos números de 5.000 cifras cuando se le da al umbral cualquier valor entre 40 y 78, puesto que cualquiera de estos valores para el umbral hace que la recursividad se detenga cuando los subcasos llegan al tamaño 40, bajando desde el tamaño 79, en el séptimo nivel de recursión. Sin embargo, estos umbrales no son equivalentes en general, porque los números de 41 cifras requieren un 17% más de tiempo para multiplicarlos con el umbral 40 frente al umbral 64. Por tanto, lo normal es que no baste con variar sencillamente el umbral para un caso de tamaño fijo. La aproximación empírica puede requerir unas cantidades notables de tiempo de computadora (¡y de tiempo humano!). En cierta ocasión, pedimos a los alumnos de un curso de algoritmia que implementasen el algoritmo divide y vencerás para multiplicar enteros muy grandes, con objeto de compararlo con el algoritmo clásico. Varios grupos intentaron estimar empíricamente el umbral óptimo, y cada grupo invirtió en el intento ¡más de 5.000 dólares de tiempo de computadora! Por otra parte, un cálculo puramente teórico del umbral óptimo no suele ser posible, dado que varía de unas implementaciones a otras.

El enfoque híbrido, que es el que nosotros recomendamos, consiste en determinar teóricamente la forma de las ecuaciones de recurrencia, y buscar entonces empíricamente los valores de las constantes que se utilizan en estas ecuaciones para la implementación concreta que estemos empleando. El umbral óptimo se pue-

de estimar entonces hallando el tamaño n del caso para el cual no hay diferencia entre aplicar directamente el algoritmo clásico o pasar a un nivel más de recursión; véase el problema 7.8. Por esta razón seleccionamos $n_0 = 64$: el algoritmo clásico de multiplicación requiere $h(64) = 64^2 = 4096$ microsegundos para multiplicar dos números de 64 cifras, mientras que si utilizamos un nivel de recursividad en el enfoque de divide y vencerás, esta misma multiplicación requiere $g(64) = 16 \times 64 = 1024$ microsegundos, además de tres multiplicaciones de números de 32 cifras mediante el algoritmo clásico, con un coste de $h(32) = 32^2 = 1.024$ microsegundos cada una, lo cual da el mismo total $3h(32) + g(64) = 4.096$ microsegundos.

Con esta técnica híbrida surge una dificultad práctica. Aun cuando el algoritmo clásico de multiplicación requiere un tiempo cuadrático, era una simplificación excesiva afirmar que $h(n) = cn^2$ para alguna constante c que depende de la implementación. Es más probable que existan tres constantes a , b y c tales que $h(n) = cn^2 + bn + a$. Aunque $bn + a$ resulta despreciable en comparación con cn^2 cuando n es grande, el algoritmo clásico se utiliza de hecho *precisamente* en casos de tamaño moderado. Por tanto, suele ser insuficiente estimar meramente el valor de la constante de orden más alto c . En lugar de hacer esto, es necesario medir $h(n)$ varias veces para distintos valores de n con objeto de estimar todas las constantes necesarias. El mismo comentario es aplicable a $g(n)$.

7.3 BÚSQUEDA BINARIA

La búsqueda binaria precedió a las computadoras. En esencia, es el algoritmo que se emplea para buscar una palabra en un diccionario, o un nombre en un directorio telefónico. Probablemente se trate de la aplicación más sencilla de divide y vencerás, tan sencilla que hablando con propiedad esto es una aplicación de *reducción (simplificación)* más que de divide y vencerás: la solución de todo caso suficientemente grande se reduce a un caso más pequeño, en este caso de tamaño mitad.

Sea $T[1..n]$ una matriz ordenada por orden no decreciente; esto es, $T[i] \leq T[j]$ siempre que sea $1 \leq i \leq j \leq n$. Sea x un elemento. El problema consiste en buscar x en la matriz T , si es que está. Formalmente, deseamos encontrar el índice i tal que $1 \leq i \leq n + 1$ y $T[i - 1] < x \leq T[i]$, con la convención lógica consistente en que $T[0] = -\infty$ y $T[n + 1] = +\infty$ (al decir *convenio lógico* queremos indicar que estos valores no están realmente presentes en la matriz). La aproximación evidente para este problema consiste en examinar secuencialmente todos los elementos de T , hasta que o bien lleguemos al final de la matriz, o bien encontremos un elemento que no sea menor que x :

```
función secuencial( $T[1..n]$ ,  $x$ )
    {Búsqueda secuencial de  $x$  en una matriz }
    para  $i \leftarrow 1$  hasta  $n$  hacer
        si  $T[i] \geq x$  entonces devolver  $i$ 
    devolver  $n + 1$ 
```

Este algoritmo requiere claramente un tiempo que está en $\Theta(r)$, en donde r es el índice que se devuelve. Esto está en $\Omega(n)$ en el caso peor, y $O(1)$ en el caso mejor. Si suponemos que los elementos de T son diferentes, que x está realmente en algún lugar de la matriz, y que tiene la misma probabilidad de encontrarse en cualquier posición, entonces el número medio de pasadas por el bucle es $(n + 1)/2$; véase el Problema 7.9. Así pues, tanto en el caso promedio como en el caso peor, la búsqueda secuencial requiere un tiempo que está en $\Theta(n)$.

Para acelerar la búsqueda, deberíamos buscar x o bien en la primera mitad de la matriz, o bien en la segunda. Para averiguar cuál de estas búsquedas es la correcta, comparamos x con un elemento de la matriz. Sea $k = \lceil n/2 \rceil$. Si $x \leq T[k]$, entonces se puede restringir la búsqueda de x a $T[1..k]$; en caso contrario basta con buscar en $T[k+1..n]$. Para evitar comparaciones repetidas en cada llamada recursiva, es mejor verificar desde un comienzo si la respuesta es $n + 1$, esto es, si x está a la derecha de T . Obtenemos el algoritmo siguiente, que se ilustra en la figura 7.1.

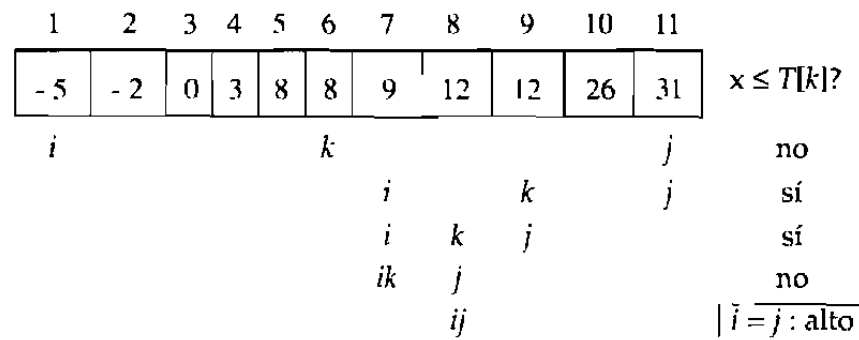


Figura 7.1. Búsqueda binaria de $x = 12$ en $T[1..11]$

función *busquedabin*($T[1..n], x$)
si $n = 0$ **o** $x > T[n]$ **entonces devolver** $n + 1$
sino devolver *binrec*($T[1..n], x$)

función *binrec*($T[i..j], x$)
 {Búsqueda binaria de x en la submatriz $T[i..j]$
 con la seguridad de que $T[i - 1] < x \leq T[j]$ }
si $i = j$ **entonces devolver** i
 $k \leftarrow (i + j) \div 2$
si $x \leq T[k]$ **entonces devolver** *binrec*($T[i..k], x$)
sino devolver *binrec*($T[k + 1..j], x$)

Sea $t(m)$ el tiempo requerido por una llamada a *binrec*($T[i..j], x$), en donde $m = j - i + 1$ es el número de elementos que restan a efectos de búsqueda. El tiempo requerido por una llamada a *busquedabin*($T[i..j], x$) es claramente $t(n)$ salvo por una pequeña constante aditiva.

Cuando $m > 1$, el algoritmo requiere una cantidad constante de tiempo además de una llamada recursiva con $\lceil m/2 \rceil$ o $\lfloor m/2 \rfloor$ elementos, dependiendo de si

$x \leq T[k]$ o no. Por tanto, $t(m) = t(m/2) + g(m)$ cuando m es par, y donde $g(m) \in O(1) = O(m^0)$. Por nuestro análisis general de los algoritmos de divide y vencerás, utilizando la ecuación 7.1 con $l = 1, b = 2$ y $k = 0$, concluimos que $t(m) \in \Theta(\log m)$. Por tanto, la búsqueda binaria se puede efectuar en un tiempo logarítmico en el caso peor. Resulta fácil ver que esta versión de la búsqueda binaria también requiere un tiempo logarítmico aun en el caso mejor.

Dado que la llamada recursiva *está dinámicamente* al final del logaritmo, es fácil realizar una versión iterativa.

```
función biniter( $T[1..n]$ ,  $x$ )
    {búsqueda binaria iterativa de  $x$  en la matriz  $T$ }
    si  $x > T[n]$  entonces devolver  $n + 1$ 
     $i \leftarrow 1; j \leftarrow n$ 
    mientras  $i < j$  hacer
        { $T[i - 1] < x \leq T[j]$ }
         $k \leftarrow (i + j) \div 2$ 
        si  $x \leq T[k]$  entonces  $j \leftarrow k$ 
        sino  $i \leftarrow k + 1$ 
    devolver  $i$ 
```

El análisis de este algoritmo es idéntico al de su contrapartida recursiva, *busqueda bin*. Se comprueban exactamente las mismas posiciones de la matriz (salvo cuando $n = 0$; véase el Problema 7.10), y los índices i , j y k asumen exactamente la misma secuencia de valores. Por tanto, la búsqueda binaria iterativa también requiere un tiempo logarítmico, tanto en el caso peor como en el mejor. Este algoritmo se puede modificar para hacerlo más rápido en el caso mejor (tiempo constante) a costa de hacerlo ligeramente más lento (aunque sigue siendo logarítmico) en el caso peor, pero esto va en detrimento del comportamiento en el caso intermedio para casos muy grandes; véase el Problema 7.11.

7.4 ORDENACIÓN

Sea $T[1..n]$ una matriz de n elementos. Nuestro problema es ordenar estos elementos por orden ascendente. Ya hemos visto que el problema se puede resolver mediante *ordenación por selección* y *ordenación por inserción* (Sección 2.4), o bien mediante *ordenación por montículo* (*heapsort*) (Sección 5.7). Recordemos que el análisis en los casos peor y promedio muestra que este último método requiere un tiempo que está en $\Theta(n \log n)$, mientras que los dos métodos anteriores requieren un tiempo cuadrático. Hay varios algoritmos clásicos de ordenación que siguen el esquema de divide y vencerás. Es interesante observar lo diferentes que son: aun habiendo decidido resolver un problema por divide y vencerás, disponemos de amplio margen de maniobra para nuestra creatividad. Estudiaremos ahora dos de ellos —*ordenar por fusión* (*mergesort*) y *ordenación rápida* (*quicksort*)— y dejaremos otro más para el Capítulo 11.

7.4.1 Ordenación por fusión

El enfoque evidente de divide y vencerás para este problema consiste en descomponer la matriz T en dos partes cuyos tamaños sean tan parecidos como sea posible, ordenar estas partes mediante llamadas recursivas, y después fusionar las soluciones de cada parte, teniendo buen cuidado en mantener el orden. Para hacer esto, necesitamos un algoritmo eficiente para fusionar dos matrices ordenadas U y V en una única matriz T cuya longitud sea la suma de las longitudes de U y V . Esto se puede lograr de forma más eficiente —y más sencilla— si se dispone de espacio adicional al final de las matrices U y V , para utilizarlo como centinela. (Esta técnica sólo funciona si podemos dar al centinela un valor previamente acordado y que estemos seguros que es mayor que cualquier elemento de U y V , valor que representaremos por « ∞ »; véase el problema 7.13):

```
procedimiento fusionar( $U[1..m+1]$ ,  $V[1..n+1]$ ,  $T[1..m+n]$ )
    {Fusiona las matrices ordenadas  $U[1..m]$  y  $V[1..n]$ 
    almacenándolas en  $T[1..m+n]$ ,  $U[m+1]$  y  $V[n+1]$ 
    se utilizan como centinelas }
     $i, j \leftarrow 1$ 
     $U[m+1], V[n+1] \leftarrow \infty$ 
    para  $k \leftarrow 1$  hasta  $m + n$  hacer
        si  $U[i] < V[j]$ 
            entonces  $T[k] \leftarrow U[i]; i \leftarrow i + 1$ 
        sino  $T[k] \leftarrow V[j]; j \leftarrow j + 1$ 
```

El algoritmo de ordenación por fusión es como sigue, en donde utilizamos la ordenación por inserción (*insertar*) de la Sección 2.4 como subalgoritmo básico. Para mejorar la eficiencia, puede ser mejor que las matrices intermedias U y V sean variables globales:

```
procedimiento ordenarporfusión( $T[1..n]$ )
    si  $n$  es suficientemente pequeño entonces insertar ( $T$ )
    sino
        matriz  $U[1..1 + \lfloor n / 2 \rfloor]$ ,  $V[1..1 + \lfloor n / 2 \rfloor]$ ,
         $U[1..\lfloor n / 2 \rfloor] \leftarrow T[1..\lfloor n / 2 \rfloor]$ 
         $V[1..\lfloor n / 2 \rfloor] \leftarrow T[1 + \lfloor n / 2 \rfloor..n]$ 
        ordenarporfusión( $U[1..\lfloor n / 2 \rfloor]$ )
        ordenarporfusión( $V[1..\lfloor n / 2 \rfloor]$ )
        fusionar( $U, V, T$ )
```

La figura 7.2 muestra la forma en que funciona la ordenación por fusión.

Proc. ~~ordenar~~ $T[1..n]$
 fus.

Matriz que hay que ordenar

3	1	4	1	5	9	2	6	5	3	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

La matriz se parte en dos mitades

3	1	4	1	5	9	
2	6	5	3	5	8	9

Una llamada recursiva a *ordenar por fusión* para cada mitad

1	1	3	4	5	9	
2	3	5	5	6	8	9

Una llamada a *fusionar*

1	1	2	3	3	4	5	5	5	6	8	9	9
---	---	---	---	---	---	---	---	---	---	---	---	---

La matriz ya está ordenada

Figura 7.2. Ordenar por fusión

Este algoritmo de ordenación ilustra bien todas las características de divide y vencerás. Cuando el número de elementos que hay que ordenar es pequeño, se utiliza un algoritmo relativamente sencillo. Por otra parte, cuando está justificado por el número de elementos, *ordenar por fusión* separa el ejemplar en dos subejemplares de tamaño mitad, resuelve los dos recursivamente, y entonces combina las dos medias matrices ya ordenadas para obtener la solución del ejemplar original.

Sea $t(n)$ el tiempo requerido por este algoritmo para ordenar una matriz de n elementos. La separación de T en U y V requiere un tiempo lineal. Es fácil ver que $\text{fusionar}(U, V, T)$ también requiere un tiempo lineal. Consiguientemente, $t(n) = t(\lfloor n/2 \rfloor) + t(\lceil n/2 \rceil) + g(n)$, en donde $g(n) \in \Theta(n)$. Esta recurrencia, que pasa a ser $t(n) = 2t(n/2) + g(n)$ cuando n es par, es un caso especial de nuestro análisis general para algoritmos de divide y vencerás. La Ecuación 7.1 es aplicable con $\ell = 2$, $b = 2$ y $k = 1$. Dado que $\ell = b^k$, se aplica el segundo caso para dar $t(n) \in \Theta(n \log n)$. Por tanto, la eficiencia de *ordenar por fusión* es similar a la de *ordenación por montículo*. La ordenación por fusión puede ser ligeramente más rápida en la práctica, pero requiere una cantidad significativamente mayor de espacio para las matrices intermedias U y V . Recuerde que *ordenación por montículo* puede ordenar *in situ*, en el sentido de que solamente necesita un pequeño número constante de variables de trabajo. La ordenación por fusión también se puede implementar *in situ*, pero a costa de tal incremento de la constante oculta que esto sólo tiene un interés teórico; véase el problema 7.14.

El algoritmo de ordenación por fusión ilustra la importancia de crear subejemplares de tamaño aproximadamente igual al desarrollar algoritmos de divide y vence-

rás. Considérese la siguiente variante de *ordenarporfusión*. (La llamada muda —ficticia— a *ordenarporfusiónmala*($V[1..1]$) sólo se incluye para acentuar la similitud con el algoritmo *ordenarporfusión* original.)

```
procedimiento ordenarporfusiónmala( $T[1..n]$ )
    si  $n$  es suficientemente pequeño entonces insert( $T$ )
    sino
        matriz  $U[1..n]$ ,  $V[1..2]$ 
         $U[1..n-1] \leftarrow T[1..n-1]$ 
         $V[1] \leftarrow T[n]$ 
        ordenarporfusiónmala( $U[1..n-1]$ )
        ordenarporfusiónmala( $V[1..1]$ )
        fusionar( $U$ ,  $V$ ,  $T$ )
```

Sea $\hat{t}(n)$ el tiempo necesario para ordenar n elementos con este algoritmo modificado. Está claro que $\hat{t}(n) = \hat{t}(n-1) + \hat{t}(1) + \hat{g}(n)$, donde $\hat{g}(n) \in \Theta(n)$. Esta recurrencia da lugar a $\hat{t}(n) \in \Theta(n^2)$. Por tanto, olvidarnos de equilibrar los tamaños de los subcasos puede ser desastroso para la eficiencia de un algoritmo obtenido por divide y vencerás. De hecho, *ordenarporfusiónmala* no es más que ¡una implementación poco eficiente de la ordenación por inserción!

7.4.2 Ordenación rápida (Quicksort)

El algoritmo de ordenación inventado por Hoare, que se suele conocer con el nombre de *quicksort* u ordenación rápida, también está basado en el principio de divide y vencerás. A diferencia de *ordenar por fusión*, la mayor parte del trabajo no recursivo que hay que hacer se invierte en construir los subcasos, y no en combinar sus soluciones. Como primer paso, el algoritmo selecciona como *pivote* uno de los elementos de la matriz que haya que ordenar. A continuación, la matriz se parte a ambos lados del pivote: se desplazan los elementos de tal manera que los que sean mayores que el pivote queden a su derecha, mientras que los demás quedan a su izquierda. Si ahora las partes de la matriz que quedan a ambos lados del pivote se ordenan independientemente mediante llamadas recursivas al algoritmo, el resultado final es una matriz completamente ordenada; no hace falta un paso subsiguiente de fusión. Para equilibrar los tamaños de los dos subcasos que hay que ordenar, nos gustaría utilizar el elemento *mediana* como pivote (para una definición de la mediana, véase la Sección 7.5). Desafortunadamente, encontrar de la mediana requiere un tiempo excesivo. Por esta razón, nos limitamos a utilizar como pivote un elemento arbitrario de la matriz, y esperamos tener suerte.

El diseño de un algoritmo de partición con tiempo lineal no es tarea difícil. Sin embargo, en la práctica resulta crucial que la constante oculta sea pequeña, para que *quicksort* sea competitivo con otras técnicas de ordenación tales como *ordenación por montículo*. Supongamos que es preciso descomponer la submatriz $T[i..j]$ empleando como pivote $p = T[i]$. Una buena forma de hacer la descomposición consiste en explorar la submatriz una sola vez, pero empezando por los dos extremos. Los punte-

ros k y l se inicializan i y $j+1$ respectivamente. A continuación, se incrementa el puntero k hasta que $T[k] > p$, y se decrementa el puntero l hasta que $T[l] \leq p$. Ahora se intercambian $T[k]$ y $T[l]$. Este proceso continúa mientras sea $k < l$. Finalmente, se intercambian $T[i]$ y $T[l]$ para poner el pivote en posición correcta:

procedimiento *pivote*($T[i..j]$; var l)

{Permuta los elementos de la matriz $T[i..j]$ y proporciona un valor l tal que, al final, $i \leq l \leq j$; $T[k] \leq p$ para todo $i \leq k < l$, $T[l] = p$,

y $T[k] > p$ para todo $l < k \leq j$, en donde p es el valor inicial de $T[i]$ }

$p \leftarrow T[i]$

$k \leftarrow i$; $l \leftarrow j+1$

repetir $k \leftarrow k+1$ **hasta que** $T[k] > p$ o $k \geq j$

repetir $l \leftarrow l-1$ **hasta que** $T[l] \leq p$

mientras $k < l$ **hacer**

intercambiar $T[k]$ y $T[l]$

repetir $k \leftarrow k+1$ **hasta que** $T[k] > p$

repetir $l \leftarrow l-1$ **hasta que** $T[l] \leq p$

intercambiar $T[i]$ y $T[l]$

Y lo que sigue es el algoritmo de ordenación. Para ordenar la matriz completa, basta llamar a *quicksort*($T[1..n]$):

procedimiento *quicksort*($T[i..j]$)

{Ordena la submatriz $T[i..j]$ por orden no decreciente}

si $j - i$ es suficientemente pequeño **entonces** *insertar* ($T[i..j]$)

sino

pivote($T[i..j]$, l)

quicksort($T[i..l-1]$)

quicksort($T[l+1..j]$)

La figura 7.3 muestra cómo funcionan *pivote* y *quicksort*.

Quicksort es ineficiente si sucede que de forma sistemática, en la mayoría de las llamadas recursivas, los subejemplares $T[i..l-1]$ y $T[l+1..j]$ están fuertemente desequilibrados. En el caso peor, por ejemplo si T ya está ordenado antes de la llamada a *quicksort*, obtenemos $l = i$ en todas las ocasiones, lo cual implica una llamada recursiva a un caso de tamaño cero y otra a un caso cuyo tamaño sólo se reduce en una unidad. Esto da lugar a una recurrencia similar a la que encontrábamos en el análisis de *ordenarporfusiónmala*, la versión no equilibrada de *ordenarporfusión*. Una vez más, el tiempo de ejecución es cuadrático. Por tanto, *quicksort* requiere en el caso peor un tiempo que está en $\Omega(n^2)$ para ordenar n elementos.

Por otra parte, si la matriz que hay que ordenar se encuentra inicialmente en orden aleatorio, entonces es probable que la mayoría de los subejemplares que haya que ordenar estén suficientemente bien equilibrados. Para determinar el tiempo *promedio* que requiere *quicksort* para ordenar una matriz de n elementos, es preciso hacer una

suposición acerca de la distribución de probabilidades de los casos de n elementos. La suposición más natural es que los elementos de T son diferentes, y que todas las $n!$ permutaciones iniciales posibles de los elementos son igualmente probables. Hay que poner de manifiesto, sin embargo, que esa suposición puede no ser válida (incluso puede ser totalmente equivocada) para algunas aplicaciones, en cuyo caso el análisis siguiente no sería aplicable. Tal cosa ocurre, por ejemplo, si nuestra aplicación necesita ordenar matrices que ya estén casi totalmente ordenadas.

Matriz que hay que ordenar

3	1	4	1	5	9	2	6	5	3	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

La matriz se particiona tomando como pivote su primer elemento, $p = 3$

3	1	4	1	5	9	2	6	5	3	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Se busca el primer elemento mayor que el pivote (subrayado) y el último elemento no mayor que el pivote (superrayado)

3	1	<u>4</u>	1	5	9	2	6	5	<u>3</u>	5	8	9
---	---	----------	---	---	---	---	---	---	----------	---	---	---

Se intercambian esos elementos

3	1	3	1	5	9	2	6	5	4	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Se vuelve a explorar en ambas direcciones

3	1	3	1	<u>5</u>	9	<u>2</u>	6	5	4	5	8	9
---	---	---	---	----------	---	----------	---	---	---	---	---	---

Se intercambian

3	1	3	1	2	9	5	6	5	4	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Se explora

3	1	3	1	<u>2</u>	<u>9</u>	5	6	5	4	5	8	9
---	---	---	---	----------	----------	---	---	---	---	---	---	---

Los punteros se han cruzado (el elemento superrayado está a la izquierda del subrayado): se intercambia el pivote con el elemento superrayado.

2	1	3	1	3	9	5	6	5	4	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

La partición ya está completada

Se ordenan recursivamente las submatrices a cada lado del pivote

1	1	2	3	3	4	5	5	5	6	8	9	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Figura 7.3. quicksort (ordenación rápida)

Sea $t(n)$ el tiempo medio que requiere una llamada a $quicksort(T[i..j])$, donde $m = j - i + 1$ es el número de elementos que hay en la submatriz. En particular, $quicksort(T[1..n])$ requiere un tiempo $t(n)$ para ordenar los n elementos de la matriz. Por nuestra suposición acerca de la distribución de probabilidad de los casos, el pivote seleccionado por el algoritmo cuando se le pide ordenar $T[1..n]$ se encuentra con igual probabilidad en cualquier posición con respecto a los demás elementos de T . Por tanto, el valor de l que devuelve el algoritmo de partición después de la llamada inicial $pivot(T[1..n], l)$ puede ser cualquier entero entre 1 y n , teniendo todos los valores la misma probabilidad $1/n$. Esta operación de pivote requiere un tiempo lineal $g(n) \in \Theta(n)$. Quedan por ordenar recursivamente dos submatrices de tamaños $l - 1$ y $n - l$ respectivamente. Se puede demostrar que la distribución de probabilidad de las submatrices sigue siendo uniforme; véase el Problema 7.16. Por tanto, el tiempo medio requerido para ejecutar estas llamadas recursivas es $t(l - 1) + t(n - l)$. Como consecuencia:

$$t(n) = \frac{1}{n} \sum_{l=1}^n (g(n) + t(l - 1) + t(n - l))$$

siempre que n sea suficientemente grande para que merezca la pena emplear el enfoque recursivo. En esta fórmula, $\frac{1}{n}$ es la probabilidad de que cualquier valor dado de l entre 1 sea devuelto por la llamada de mayor nivel a $pivot$, y $g(n) + t(l - 1) + t(n - l)$ es el tiempo esperado para ordenar n elementos con la condición de que este valor de l sea devuelto por esa llamada.

Para hacer más explícita la fórmula, sea n_0 el umbral por encima del que se utiliza el enfoque recursivo, lo cual quiere decir que se utiliza la ordenación por inserción siempre que no haya más de n_0 elementos para ordenar. Además, sea d una constante (que dependerá de la implementación) tal que $g(n) \leq dn$ siempre que $n > n_0$. Si sacamos $g(n)$ del sumatorio, tenemos

$$t(n) \leq dn + \frac{2}{n} \sum_{l=1}^n (t(l - 1) + t(n - l)) \quad \text{para } n > n_0$$

Si observamos que el término $t(k)$ aparece dos veces por cada $0 \leq k \leq n - 1$, una sencilla manipulación da lugar a

$$t(n) \leq dn + \frac{2}{n} \sum_{k=0}^{n-1} t(k) \quad \text{para } n > n_0 \quad (7.2)$$

Una ecuación de este tipo es más difícil de analizar que las recurrencias lineales que vimos en la Sección 4.7. En particular, la ecuación 7.1 no puede aplicarse en esta ocasión. Por analogía con *ordenar por fusión*, resulta sin embargo razonable esperar que $t(n)$ esté en $O(n \log n)$: en término medio, los subejemplares no están demasiado desequilibrados y la solución estaría en $O(n \log n)$ si estuvieran tan bien equilibra-

dos como con *ordenar por fusión*. La demostración de esta conjetura es una bonita aplicación de la técnica de inducción constructiva (Sección 1.6.4). Para aplicar esta técnica, postulamos la existencia de una constante c , desconocida por el momento, tal que $t(n) \leq c n \log n$ para todo $n \geq 2$. Hallaremos un valor adecuado para esta constante en el proceso de demostración de su existencia por inducción matemática generalizada. Comenzamos con $n = 2$, porque $n \log n$ está indefinido o es nulo para valores más pequeños de n ; alternatively, podríamos empezar en $n = n_0 + 1$.

Teorema 7.4.1 *Quicksort requiere un tiempo en $O(n \log n)$ para ordenar n elementos en el caso medio.*

Demostración. Sea $t(n)$ el tiempo requerido por *quicksort* para ordenar n elementos en el caso promedio. Sean d y n_0 constantes tales que es válida la ecuación 7.2. Deseamos demostrar que $t(n) \leq c n \log n$ para todo $n \geq 2$, siempre y cuando c sea una constante elegida adecuadamente. Procedemos por inducción constructiva. Supongamos sin pérdida de generalidad que $n_0 \geq 2$.

◊ *Base:* consideremos cualquier entero n tal que $2 \leq n \leq n_0$. Tenemos que demostrar que $t(n) \leq c n \log n$. Esto es fácil, puesto que tenemos total libertad para seleccionar la constante c , y el número de casos es finito. Basta con seleccionar c que sea al menos tan grande como $t(n) / (n \log n)$. De esta manera, nuestra primera restricción sobre c es:

$$c \geq \frac{t(n)}{n \log n} \text{ para todo } n \text{ tal que } 2 \leq n \leq n_0 \quad (7.3)$$

◊ *Paso de inducción:* consideremos cualquier entero $n > n_0$. Se toma como hipótesis de inducción que $t(k) \leq c k \log k$ para todo k tal que $2 \leq k < n$. Deseamos restringir c de tal manera que $t(n) \leq c n \log n$ se siga de la hipótesis de inducción. Supongamos que a representa $t(0) + t(1)$. Comenzando con la ecuación 7.2:

$$\begin{aligned} t(n) &\leq dn + \frac{2}{n} \sum_{k=0}^{n-1} t(k) \\ &= dn + \frac{2}{n} \left(t(0) + t(1) + \sum_{k=2}^{n-1} t(k) \right) \\ &\leq dn + \frac{2a}{n} + \frac{2}{n} \sum_{k=2}^{n-1} ck \log k \text{ por la hipótesis de inducción} \\ &\leq dn + \frac{2a}{n} + \frac{2c}{n} \int_2^n x \log x dx \text{ (véase la figura 7.4)} \end{aligned}$$

$$= dn + \frac{2a}{n} + \frac{2c}{n} \left[\frac{x^2 \log x}{2} - \frac{x^2}{4} \right]_{x=2}^n$$

(recordemos que «log» representa el logaritmo natural)

$$< dn + \frac{2a}{n} + \frac{2c}{n} \left(\frac{n^2 \log n}{2} - \frac{n^2}{4} \right)$$

$$= dn + \frac{2a}{n} cn \log n - \frac{cn}{2}$$

$$= cn \log n - \left(\frac{c}{2} - d - \frac{2a}{n^2} \right) n.$$

Se sigue que $t(n) \leq c n \log n$ siempre y cuando $c/2 - d - 2a/n^2$ sea no negativo, lo cual equivale a decir que $c \geq 2d + 4a/n^2$. Dado que aquí consideramos solamente el caso en el cual $n > n_0$, todo va bien siempre y cuando

$$c \geq 2d + \frac{4a}{(n_0 + 1)^2} \quad (7.4)$$

que es nuestra segunda y última restricción sobre c .

Reuniendo las restricciones dadas por las ecuaciones 7.3 y 7.4, basta hacer

$$c = \max \left(2d + \frac{4(t(0) + t(1))}{(n_0 + 1)^2}, \max \left\{ \frac{t(n)}{n \log n} \mid 2 \leq n \leq n_0 \right\} \right) \quad (7.5)$$

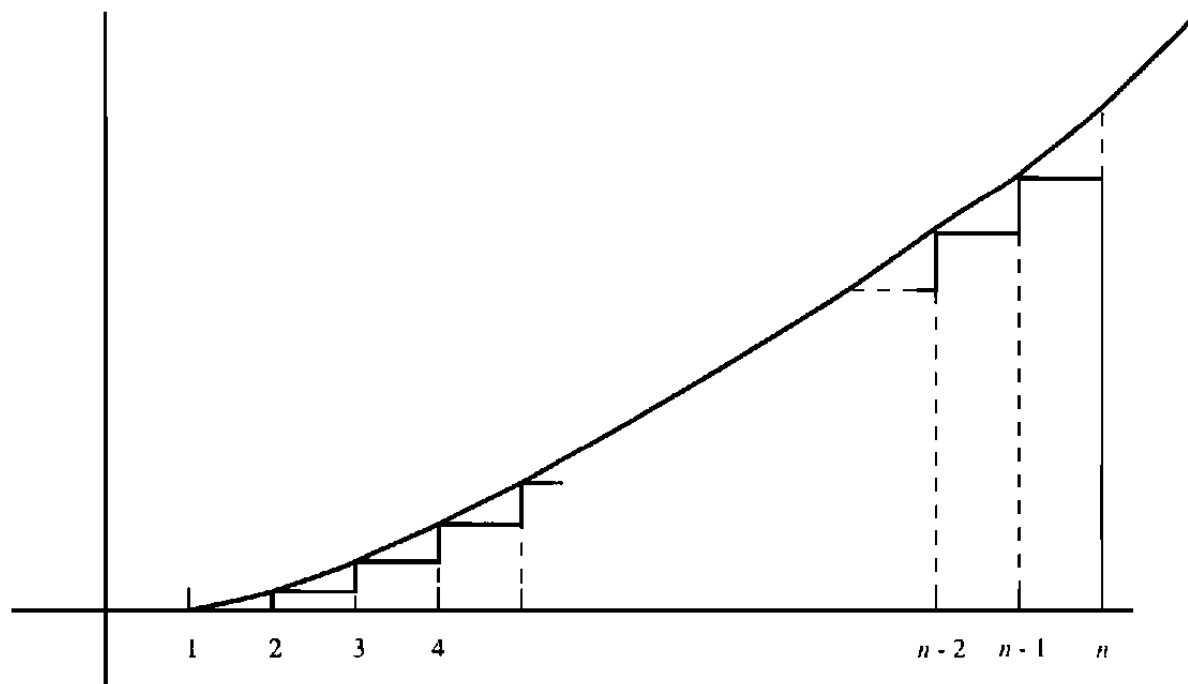


Figura 7.4. Suma de funciones monótonas

para finalizar la demostración por inducción constructiva de que $t(n) \leq c n \log n$ para todo $n \geq 2$, y por tanto que $t(n) \in O(n \log n)$.

Si se siente confuso o poco convencido, le animamos a que desarrolle por sí mismo una demostración por inducción matemática generalizada —en vez de inducción matemática constructiva— de que $t(n) \leq c n \log n$ para todo $n \geq 2$, utilizando esta vez el valor explícito de c dado por la ecuación 7.5.

Por tanto, *quicksort* puede ordenar una matriz de n elementos distintos en un tiempo promedio que está en $O(n \log n)$. En la práctica, la constante oculta es más pequeña que las asociadas en *ordenación por montículo* o en *ordenar por fusión*. Si ocasionalmente se puede tolerar algún tiempo de ejecución largo, éste es un excelente algoritmo de propósito general. ¿Se puede modificar *quicksort* para que requiera un tiempo que esté en $O(n \log n)$ incluso en el caso peor? La respuesta es sí pero no. Incluso si seleccionamos la mediana de $T[i..j]$ como pivote, lo cual se puede hacer en un tiempo lineal según se verá en la Sección 7.5, *quicksort* tal como se ha descrito aquí sigue requiriendo un tiempo cuadrático en el caso peor, lo cual sucede si son iguales todos los elementos que hay que ordenar. Basta una sencilla modificación del algoritmo de partición para evitar este comportamiento incorrecto, aunque es complicado programarlo de forma correcta y eficiente. Para esto, se necesita un nuevo

procedimiento *pivotebis*($T[i..j]$, p ; var k, l)

que descomponga T en tres secciones, empleando p como pivote: después de la partición, los elementos de $T[i..k]$ son más pequeños que p , los de $T[k+1..l-1]$ son iguales a p , y los de $T[l..j]$ son más grandes que p . Los valores de k y l son devueltos por *pivotebis*. Después de hacer la partición con una llamada a *pivotebis*($T[i..j]$, $T[i]$, k , l) hay que llamar a *quicksort* recursivamente, con $T[i..k]$ y $T[l..j]$. Con esta modificación, la ordenación de una matriz de elementos iguales requiere un tiempo lineal. Hay algo más interesante: ahora *quicksort* requiere un tiempo que está en $O(n \log n)$ incluso en el caso peor si se selecciona como pivote la mediana de $T[i..j]$ en un tiempo lineal. Sin embargo, sólo mencionamos esta posibilidad para indicar que hay que evitarla: la constante oculta asociada a esta versión «mejorada» de *quicksort* es tan grande que da lugar a un algoritmo, peor que *ordenación por montículo* en todos los casos.

7.5 BÚSQUEDA DE LA MEDIANA

Sea $T[1..n]$ una matriz de enteros, y sea s un entero entre 1 y n . Se define el s -ésimo menor elemento de T como aquel elemento que se encontraría en la s -ésima posición si se ordenara T en orden no decreciente. Dados T y s , el problema de encontrar el s -ésimo elemento de T se conoce como el *problema de selección*. En particular, se define la *mediana* de T como su $\lceil n/2 \rceil$ -ésimo elemento. Cuando n es

impar y los elementos de T son diferentes, la mediana es simplemente aquel elemento de T tal que en T hay tantos elementos más pequeños que él como elementos mayores que él. Por ejemplo, la mediana de $[3, 1, 4, 1, 5, 9, 2, 6, 5]$ es 4, puesto que 3, 1, 1 y 2 son más pequeños que 4, mientras que 5, 9, 6 y 5 son mayores.

¿Qué podría ser más sencillo que buscar el menor elemento de T , o calcular la media de todos los elementos? Sin embargo, no es evidente que sea posible calcular la mediana con tanta facilidad. Un algoritmo sencillo para determinar la mediana de $T[1..n]$ consiste en ordenar la matriz y extraer entonces el elemento $\lceil n/2 \rceil$ -ésimo. Si utilizamos *ordenación por montículo* u *ordenación por fusión*, esto requiere un tiempo que está en $\Theta(n \log n)$. ¿Podemos hacerlo mejor? Para responder a esta pregunta, estudiaremos la interrelación entre hallar la mediana y seleccionar el s -ésimo elemento más pequeño.

Es evidente que todo algoritmo para el problema de selección se puede utilizar para hallar la mediana: basta con seleccionar el $\lceil n/2 \rceil$ -ésimo elemento más pequeño. Curiosamente, la inversa también es cierta. Supongamos por el momento que está disponible un algoritmo *mediano*($T[1..n]$) que devuelve la mediana de T . Dada una matriz T y un entero s , ¿cómo se podría utilizar este algoritmo para determinar el s -ésimo elemento más pequeño de T ? Sea p la mediana de T . Ahora usamos p como pivote, de forma muy parecida a *quicksort*, pero usando el algoritmo *pivotebis* presentado al final de la sección anterior. Recuerde que una llamada a *pivotebis*($T[1..n], p; \text{var } k, l$) particiona a $T[i..j]$ en tres secciones: T se reorganiza de tal manera que los elementos de $T[i..k]$ sean más pequeños que p , los de $T[k+1..l-1]$ sean iguales a p , y los de $T[l..j]$ sean mayores que p . Tras una llamada a *pivotebis*(T, p, k, l), hemos terminado si $k < s < l$, puesto que entonces el s -ésimo elemento más pequeño de T es igual a p . Si $s \leq k$, entonces el s -ésimo elemento más pequeño de T es ahora el s -ésimo elemento más pequeño de $T[1..k]$. Por último, si $s \geq l$, entonces el s -ésimo elemento más pequeño de T es ahora el $(s - l + 1)$ -ésimo elemento más pequeño de $T[l..n]$. En cualquier caso, hemos avanzado, porque o bien hemos terminado, o bien la submatriz que hay que considerar contiene menos de la mitad de los elementos, por cuanto p es la mediana de la matriz original.

Hay una gran similitud entre este enfoque y la búsqueda binaria (Sección 7.3), y de hecho el algoritmo resultante se puede programar de forma iterativa en lugar de hacerlo recursivamente. La idea clave consiste en emplear dos variables i y j , inicializadas a 1 y n respectivamente, y asegurarnos que en todo momento $i \leq s \leq j$, y que los elementos de $T[1..i-1]$ sean más pequeños que los de $T[l..j]$, que a su vez son más pequeños que los de $T[j+1..n]$. La consecuencia inmediata es que el elemento deseado se encuentra en $T[i..j]$. Cuando todos los elementos de $T[i..j]$ sean iguales, hemos acabado.

La figura 7.5 ilustra el proceso. Por sencillez, la ilustración supone que *pivotebis* se ha implementado de una manera que es intuitivamente sencilla, aun cuando una implementación realmente eficiente haría las cosas de otra manera.

```
función selección( $T[1..n]$ ,  $s$ )
  {Busca el  $s$ -ésimo elemento más pequeño de  $T$ ,  $1 \leq s \leq n$ }
   $i \leftarrow 1$ ;  $j \leftarrow n$ 
  repetir
    {La respuesta se encuentra en  $T[i..j]$ }
     $p \leftarrow \text{mediana}(T[i..j])$ 
     $\text{pivotebis}(T[i..j], p, k, l)$ 
    si  $s \leq k$  entonces  $j \leftarrow k$ 
    sino si  $s \geq l$  entonces  $i \leftarrow l$ 
    sino devolver  $p$ 
```

Matriz en la que hay que hallar el 4.º elemento más pequeño

3	1	4	1	5	9	2	6	5	3	5	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Se particiona la matriz usando como pivote $p = 5$, con *pivotebis*

3	1	4	1	2	3	5	5	5	9	6	8	9
---	---	---	---	---	---	---	---	---	---	---	---	---

Sólo es relevante la parte a la izquierda del pivote, puesto que $4 \leq 7$

3	1	4	1	2	3	.	.	.	.	.	.	.
---	---	---	---	---	---	---	---	---	---	---	---	---

Se particiona esa parte usando como pivote su mediana $p = 2$

1	1	2	3	4	3	.	.	.	.	.	.	.
---	---	---	---	---	---	---	---	---	---	---	---	---

Sólo es relevante la parte a la derecha del pivote puesto que $4 \geq 4$

.	.	.	3	4	3	.	.	.	.	.	.	.
---	---	---	---	---	---	---	---	---	---	---	---	---

Se particiona esa parte usando como pivote su mediana $p = 3$

.	.	.	3	3	4	.	.	.	.	.	.	.
---	---	---	---	---	---	---	---	---	---	---	---	---

La respuesta es 3, porque el pivote está en la 4.ª posición

Figura 7.5. Selección empleando la mediana

Mediante un análisis similar al de la búsqueda binaria, el algoritmo anterior selecciona el elemento requerido de T pasando por el bucle **repetir** un número logarítmico de veces en el caso peor. Sin embargo, las pasadas por el bucle ya no requieren un tiempo constante y lo cierto es que este algoritmo no se podrá utilizar

mientras no dispongamos de una forma eficiente de hallar la mediana, que era nuestro problema original. ¿Podemos modificar el algoritmo para evitar recurrir a la mediana?

En primer lugar, observemos que nuestro algoritmo sigue funcionando independientemente del elemento de T que seleccionemos como pivote (el valor de p). Solamente la *eficiencia* del algoritmo depende de la elección de pivote: el uso de la mediana asegura que el número de elementos que estamos considerando se divide al menos por dos en cada pasada por el bucle **repetir**. Si estamos dispuestos a sacrificar la velocidad en el caso peor para obtener un algoritmo razonablemente rápido de análisis en el caso promedio, podemos tomar prestada otra idea de *quicksort*, y seleccionar simplemente $T[i]$ como pivote. En otras palabras, sustituimos la primera instrucción del bucle por

$$p \leftarrow T[i]$$

Esto da lugar a que el algoritmo invierta un tiempo cuadrático en el caso peor, por ejemplo si la matriz está en orden decreciente y deseamos encontrar el elemento más pequeño. Sin embargo, este algoritmo modificado funciona en un tiempo *lineal* de análisis en el caso promedio, con la suposición habitual de que los elementos de T sean distintos y que las $n!$ permutaciones de los elementos iniciales posibles sean igualmente probables. (El análisis es paralelo al de *quicksort*, véase el problema 7.18.) Esto es mucho mejor que el tiempo que se necesita de análisis en el caso promedio si realmente ordenamos la matriz, pero el comportamiento en el caso peor no es admisible para muchas aplicaciones.

Felizmente, este caso peor cuadrático se puede evitar sin sacrificar el comportamiento lineal de análisis en el caso promedio. La idea es que el número de pasadas por el bucle sigue siendo logarítmica siempre y cuando seleccionemos un pivote razonablemente próximo a la mediana. Una buena *aproximación* a la mediana se puede hallar rápidamente con un poco de astucia. Consideremos el algoritmo siguiente:

```
función pseudomediana( $T[1..n]$ )
  {Busca una aproximación a la mediana de la matriz  $T$ }
  si  $n \leq 5$  entonces devolver medianadhoc( $T$ )
   $z \leftarrow \lfloor n/5 \rfloor$ 
  matriz  $Z[1..z]$ 
  para  $i \leftarrow 1$  hasta  $z$  hacer  $Z[i] \leftarrow \text{medianadhoc}(T[5i-4..5i])$ 
  devolver selección( $Z, \lfloor z/2 \rfloor$ )
```

Aquí, *medianadhoc* es un algoritmo diseñado especialmente para hallar la mediana de un máximo de 5 elementos, lo cual se puede hacer en un tiempo limitado superiormente por una constante, y *selección*($Z, \lfloor z/2 \rfloor$) determina la mediana exacta de la matriz Z . Sea p el valor devuelto por una llamada a *pseudomediana*(T). ¿A qué distancia de la verdadera mediana de T puede estar p cuando $n > 5$?

Tal como en el algoritmo, sea z igual a $\lfloor n/5 \rfloor$, el número de elementos de la matriz Z creada por la llamada a *pseudomediana*(T). Para todo i entre 1 y z , $Z[i]$ es por definición la mediana de $T[5i - 4..5i]$ y por tanto al menos tres elementos de los cinco que hay en la matriz son menores o iguales que él. Además, como p es la verdadera mediana de Z , al menos $z/2$ elementos de Z son menores o iguales que p . Por transitividad, $(T[j] \leq Z[i] \leq p \text{ implica que } T[j] \leq p)$, al menos $3z/2$ elementos de T son menores o iguales que p . Dado que es $z = \lfloor n/5 \rfloor \geq (n-4)/5$, concluimos que al menos $(3n-12)/10$ elementos de T son menores o iguales que p , y por tanto como mucho los $(7n+12)/10$ elementos restantes de T son estrictamente mayores que p . El mismo razonamiento se aplica al número de elementos de T que son estrictamente *más pequeños* que p .

Aun cuando es probable que p no sea la mediana exacta de T , concluimos que su rango está aproximadamente entre $3n/10$ y $7n/10$. Para visualizar la forma en que surgen estos factores, aunque no hay nada en la ejecución de *pseudomediana* que realmente se corresponda con esta ilustración, consideremos cómo en la figura 7.6 que los elementos de T están dispuestos en cinco filas, con la posible excepción de un máximo de 4 elementos, que apartamos. Supongamos ahora que todas las $\lfloor n/5 \rfloor$ columnas, así como la fila del medio se ordenan, por arte de magia, con los elementos más pequeños situados en la parte superior y en el lado izquierdo respectivamente. La fila del medio corresponde a la matriz Z del algoritmo, y el elemento rodeado por un círculo correspondiendo a la mediana de esta matriz, que es el valor de p devuelto por el algoritmo. Claramente, cada uno de los elementos del cuadro es menor o igual que p . La conclusión se sigue automáticamente, puesto que el cuadro contiene aproximadamente entre tres quintos y la mitad de los elementos de T .

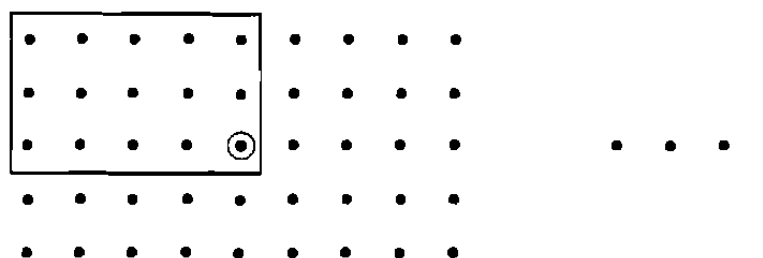


Figura 7.6. Visualización de la pseudomediana

Ahora analizaremos la eficiencia del algoritmo de *selección* presentado al principio de esta sección, cuando se sustituye la primera instrucción del bucle **repetir** por

$$p \leftarrow \text{pseudomediana}(T[i..j])$$

Sea $t(n)$ el tiempo requerido en el caso peor por una llamada a *selección*($T[1..n]$, s). Consideremos i y j cualesquiera, tales que $1 \leq i \leq j \leq n$. El tiempo requerido para completar el bucle **repetir** con estos valores de i y j es esencialmente $t(m)$, donde $m = j - i + 1$ es el número de elementos que quedan por considerar. Cuando $n > 5$, el cálculo de *pseudomediana*(T) requiere un tiempo que está en $t(\lfloor n/5 \rfloor) + O(n)$,

porque la matriz Z se puede construir en tiempo lineal, dado que cada llamada a *medianadhoc* requiere un tiempo constante. La llamada a *pivotebis* también requiere un tiempo lineal. Llegado este momento, o bien hemos terminado o bien hay que volver a pasar por el bucle con un máximo de $(7n + 12)/10$ elementos restantes por considerar. Por tanto, existe una constante d tal que

$$t(n) \leq dn + t(\lfloor n/5 \rfloor) + \max \{t(m) \mid m \leq (7n + 12)/10\} \quad (7.6)$$

siempre y cuando $n > 5$.

La ecuación 7.1 no nos ayuda a resolver esta recurrencia, así que una vez más recurriremos a la inducción constructiva. Esta vez es precisa una cierta perspicacia incluso para hacer una suposición acerca de la respuesta en notación asintótica. (Recordemos que la propuesta evidente cuando analizábamos *quicksort* era $O(n \log n)$ porque ya habíamos analizado *ordenar por fusión*, y funcionaba; esta vez no tenemos tanta suerte.) Con una cierta experiencia, el hecho de que $\frac{1}{5} + \frac{7}{10} < 1$ es una indicación de que $t(n)$ puede muy bien ser lineal en n , lo cual es claramente lo mejor que podemos esperar; véase el problema 7.19.

Teorema 7.5.1 *El algoritmo de selección empleado con pseudomediana halla el s -ésimo elemento más pequeño entre n , con un tiempo que está en $\Theta(n)$ en el caso peor. En particular, la mediana se puede hallar en un tiempo lineal en el caso peor.*

Demostración. Sean $t(n)$ y d igual que antes. Claramente, $t(n) \in \Omega(n)$, porque el algoritmo tiene que examinar todos y cada uno de los elementos de T al menos una vez. Por tanto, queda por demostrar que $t(n) \in O(n)$. Postulemos la existencia de una constante c , desconocida por el momento, tal que $t(n) \leq cn$ para todo $n \geq 1$. Encontraremos un valor adecuado para esta constante en el proceso de demostración de su existencia por inducción matemática generalizada. También se utilizará la inducción constructiva para determinar la constante n_0 que separa el caso base del paso de inducción. Por el momento, nuestra única restricción es que $n_0 \geq 5$, porque la ecuación 7.6 solamente es aplicable cuando $n > 5$ (descubriremos que la elección evidente, $n_0 = 5$, no es válida).

◇ *Base:* consideremos cualquier entero n tal que $1 \leq n \leq n_0$. Tenemos que mostrar que $t(n) \leq cn$. Esto es sencillo, porque tenemos total libertad para seleccionar la constante c , y el número de casos es finito. Basta con seleccionar c al menos tan grande como $t(n)/n$. Por tanto, nuestra primera restricción sobre c es:

$$c \geq t(n)/n \text{ para todo } n \text{ tal que } 1 \leq n \leq n_0 \quad (7.7)$$

◇ *Paso de inducción:* consideremos cualquier entero $n > n_0$. Como hipótesis de inducción, supongamos que $t(m) \leq cm$ cuando $1 \leq m < n$. Deseamos restringir c de tal forma que $t(n) \leq cn$ se siga de la hipótesis de inducción. Comenzando por la ecuación 7.6, y dado que $1 \leq (7n + 12)/10$ cuando $n > n_0 \geq 5$:

$$\begin{aligned} t(n) &\leq dn + t(\lfloor n/5 \rfloor) + \max\{t(m) \mid m \leq (7n + 12)/10\} \\ &\leq dn + cn/5 + (7n + 12)c/10 \text{ por la hipótesis de inducción} \\ &= 9cn/10 + dn + 6c/5 \\ &= cn - (c/10 - d - 6c/5n)n \end{aligned}$$

Se sigue que $t(n) \leq cn$ siempre y cuando $c/10 - d - 6c/5n \geq 0$, lo cual es equivalente a $(1 - 12/n)c \geq 10d$. Esto es posible siempre y cuando sea $n \geq 13$ (para que $1 - 12/n > 0$), en cuyo caso c no debe de ser menor que $10d/(1 - 12/n)$. Teniendo en cuenta que $n > n_0$, cualquier elección de $n_0 \geq 12$ será aceptable, siempre y cuando c se seleccione en consecuencia. Más exactamente, todo va bien siempre y cuando $n_0 \geq 12$ y

$$c \geq \frac{10d}{1 - \frac{12}{n_0 + 1}} \quad (7.8)$$

que es nuestra segunda y última restricción sobre c y n_0 . Por ejemplo, el paso de inducción es correcto si tomamos $n_0 = 12$ y $c \geq 130d$, o bien $n_0 = 23$ y $c \geq 20d$, o bien $n_0 = 131$ y $c \geq 11d$.

Reuniendo las restricciones dadas por las ecuaciones 7.7 y 7.8, y seleccionando $n_0 = 23$ para concretar, basta con hacer

$$c = \max(20d, \max\{t(m)/m \mid 1 \leq m \leq 23\})$$

para finalizar la demostración por inducción constructiva de que $t(n) \leq cn$ para todo $n \geq 1$.

7.6 MULTIPLICACIÓN DE MATRICES

Sean A y B dos matrices $n \times n$ que hay que multiplicar, y sea C su producto. El algoritmo clásico de multiplicación de matrices proviene directamente de su definición:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

Cada entrada de C se calcula en un tiempo que está en $\Theta(n)$, suponiendo que la suma y la multiplicación de escalares sean operaciones elementales. Dado que hay que calcular n^2 entradas, el producto AB se puede calcular en un tiempo que está en $\Theta(n^3)$.

Hacia el final de los años 60, Strassen dio lugar a un revuelo considerable, al mejorar este algoritmo. Desde un punto de vista algorítmico, este descubrimiento

fue un hito en la historia de «divide y vencerás», aun cuando el algoritmo no menos sorprendente para multiplicar enteros muy grandes (Sección 7.1) se hubiera descubierto casi una década antes. La idea básica del algoritmo de Strassen es similar a la anterior. En primer lugar, mostramos que las matrices 2×2 se pueden multiplicar empleando menos multiplicaciones escalares que las ocho que requiere aparentemente la definición. Sean

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \text{ y } B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

las dos matrices que hay que multiplicar. Consideremos las operaciones siguientes, cada una de las cuales necesita una única multiplicación:

$$\left. \begin{aligned} m_1 &= (a_{21} + a_{22} - a_{11}) (b_{22} - b_{12} + b_{11}) \\ m_2 &= a_{11} b_{11} \\ m_3 &= a_{12} b_{21} \\ m_4 &= (a_{11} - a_{21}) (b_{22} - b_{12}) \\ m_5 &= (a_{21} + a_{22}) (b_{12} - b_{11}) \\ m_6 &= (a_{12} - a_{21} + a_{11} - a_{22}) b_{22} \\ m_7 &= a_{22} (b_{11} + b_{22} - b_{12} - b_{21}) \end{aligned} \right\} \quad (7.9)$$

Dejaremos al lector verificar que el producto matricial AB que se pide está dado por la matriz siguiente.

$$C = \begin{pmatrix} m_2 + m_3 & m_1 + m_2 + m_5 + m_6 \\ m_1 + m_2 + m_4 - m_7 & m_1 + m_2 + m_4 + m_5 \end{pmatrix} \quad (7.10)$$

Por tanto, es posible multiplicar dos matrices 2×2 empleando solamente siete multiplicaciones escalares. A primera vista, este algoritmo no parece muy interesante: utiliza un elevado número de sumas y restas en comparación con las cuatro sumas que bastan en el algoritmo clásico.

Si ahora sustituimos cada entrada de A y B por una matriz $n \times n$, obtenemos un algoritmo que puede multiplicar dos matrices $2n \times 2n$ efectuando siete multiplicaciones de matrices $n \times n$, así como un cierto número de sumas y restas de matrices $n \times n$. Esto es posible porque el algoritmo 2×2 no se basa en la conmutatividad de la multiplicación escalar. Dado que es mucho más rápido sumar matrices grandes que multiplicarlas, el ahorro de una multiplicación compensa sobradamente las sumas adicionales.

Sea $t(n)$ el tiempo necesario para multiplicar dos matrices $n \times n$ mediante el uso recursivo de las ecuaciones 7.9 y 7.10. Supongamos por sencillez que n es una potencia de 2. Dado que las matrices se pueden sumar y restar en un tiempo que está en $\Theta(n^2)$, $t(n) = 7t(n/2) + g(n)$, en donde $g(n) \in \Theta(n^2)$. Esta recurrencia es otro ejemplo de nuestro análisis general de los algoritmos divide y vencerás. La ecuación 7.1 es aplicable con $\ell = 7$, $b = 2$ y $k = 2$. Dado que $\ell > b^k$, el tercer caso produce $t(n) \in \Theta(n^{\lg 7})$. Las matrices cuadradas cuyo tamaño no sea una potencia de 2 se pueden tratar fácilmente añadiéndoles filas y columnas de ceros, y doblando co-

mo mucho su tamaño, lo cual no afecta al tiempo asintótico de ejecución. Como $\lg 7 < 2,81$, resulta entonces posible multiplicar dos matrices en un tiempo que está en $O(n^{2.81})$, siempre y cuando las operaciones escalares sean elementales.

Siguiendo el descubrimiento de Strassen, un cierto número de investigadores ha intentado mejorar la constante ω de tal manera que sea posible multiplicar dos matrices $n \times n$ en un tiempo que esté en $O(n^\omega)$. Lo primero que se podía intentar, evidentemente, era multiplicar dos matrices 2×2 empleando *seis* multiplicaciones escalares. Pero en 1971, Hopcroft y Kerr demostraron que esto era imposible cuando no se puede emplear la conmutatividad de la multiplicación. Lo siguiente que había que intentar era encontrar una forma de multiplicar dos matrices 3×3 empleando como máximo 21 multiplicaciones escalares. Esto daría lugar a un algoritmo recursivo para multiplicar matrices $n \times n$ en un tiempo que estaría en $\Theta(n^{\log_3 21})$, que es asintóticamente más rápido que el algoritmo de Strassen, porque $\log_3 21 < \log_2 7$. Desafortunadamente, esto también es imposible.

Pasó casi una década antes de que Pan descubriera una forma de multiplicar dos matrices 70×70 empleando 143.640 multiplicaciones escalares —compárese esto con las 343.000 que requiere el algoritmo clásico— y de hecho $\log_{70} 143.640$ es un poquitín más pequeño que $\lg 7$. Este descubrimiento dio pie a la denominada *guerra de los decimales*. Se fueron descubriendo sucesivamente numerosos algoritmos cada vez más eficientes asintóticamente. Por ejemplo, al final de 1979 se sabía que era posible multiplicar matrices en un tiempo que estaba en $O(n^{2.521813})$; imagine la excitación reinante en enero de 1980 cuando esto se mejoró hasta alcanzar $O(n^{2.521801})$. El algoritmo de multiplicación matricial asintóticamente más rápido conocido en el momento de escribir esto se remonta a 1986, año en que Coppersmith y Winograd descubrieron que es posible, al menos en teoría, multiplicar dos matrices $n \times n$ en un tiempo que está en $O(n^{2.376})$. Sin embargo, y como consecuencia de las constantes ocultas, ninguno de los algoritmos hallados después del de Strassen tiene una aplicación práctica real.

7.7 EXPONENCIACIÓN

Sean a y n dos enteros. Deseamos calcular la exponenciación $x = a^n$. Por sencillez, supondremos en toda esta sección que $n > 0$. Si n es pequeño, el algoritmo evidente resulta adecuado:

función *exposec*(a, n)

$r \leftarrow a$

para $i \leftarrow 1$ **hasta** $n - 1$ **hacer** $r \leftarrow a \times r$

devolver r

Este algoritmo requiere un tiempo que está en $\Theta(n)$, puesto que la instrucción $r \leftarrow a \times r$ se ejecuta exactamente $n - 1$ veces, *siempre y cuando las multiplicaciones cuenten como operaciones elementales*. Sin embargo, en la mayoría de las computadoras, hasta los valores pequeños de n y de a hacen que este algoritmo produzca desbordamientos enteros. Por ejemplo, 15^{17} no cabe en un entero de 64 *bits*.

Si deseamos manejar operandos más grandes, es preciso tener en cuenta el tiempo necesario para cada multiplicación. Sea $M(q, s)$ el tiempo necesario para multiplicar dos enteros de tamaños q y s . Para nuestros propósitos, no importa si consideramos el tamaño de los enteros en dígitos decimales, en *bits* o en cualquier otra base fija mayor que 1. Supongamos por sencillez que $q_1 \leq q_2$ y $s_1 \leq s_2$ implican que $M(q_1, s_1) \leq M(q_2, s_2)$. Estimemos cuánto tiempo invierte nuestro algoritmo multiplicando enteros cuando se invoca $\text{exposec}(a, n)$. Sea m el tamaño de a . En primer lugar, observemos que el producto de dos enteros de tamaños i y j tiene un tamaño que es al menos $i + j - 1$ y como máximo $i + j$; véase el problema 7.24. Sean r_i y m_i el valor y el tamaño de r al principio de la i -ésima pasada por el bucle. Claramente, $r_1 = a$ y por tanto $m_1 = m$. Dado que $r_{i+1} = ar_i$, el tamaño de r_{i+1} es al menos $m + m_i - 1$ y como máximo $m + m_i$. La demostración por inducción matemática de que $im - i + 1 \leq m_i \leq im$ para todo i se sigue inmediatamente. Por tanto, la multiplicación efectuada en la i -ésima pasada por el bucle afecta a un entero de tamaño m y a un entero cuyo tamaño se encuentra entre $im - i + 1$ e im , lo cual requiere un tiempo que está entre $M(m, im - i + 1)$ y $M(m, im)$. El tiempo total $T(m, n)$ que se invierte en multiplicaciones cuando se calcula a^n con exposec es, por tanto:

$$\sum_{i=1}^{n-1} M(m, im - i + 1) \leq T(m, n) \leq \sum_{i=1}^n M(i, im) \quad (7.11)$$

donde m es el tamaño de a . $T(m, n)$ es una buena estimación del tiempo total requerido por exposec , puesto que la mayor parte del trabajo se invierte en realizar estas multiplicaciones. Si utilizamos el algoritmo clásico de multiplicación (Sección 1.2), entonces $M(q, s) \in \Theta(qs)$. Sea c tal que $M(q, s) \leq cqs$:

$$\begin{aligned} T(m, n) &\leq \sum_{i=1}^{n-1} M(m, im) \leq \sum_{i=1}^{n-1} c \cdot m \cdot im \\ &= cm^2 \sum_{i=1}^{n-1} i < cm^2 n^2 \end{aligned}$$

De esta forma, $T(m, n) \in O(m^2 n^2)$. Resulta igualmente fácil demostrar, a partir de la ecuación 7.11, que $T(m, n) \in \Omega(m^2 n^2)$ y por tanto que $T(m, n) \in \Theta(m^2 n^2)$; véase el Problema 7.25. Por otra parte, si utilizamos el algoritmo de multiplicación de divide y vencerás que se describía anteriormente en este capítulo, $M(q, s) \in \Theta(sq^{\lg(3/2)})$ cuando $s \geq q$, y un argumento similar da $T(m, n) \in \Theta(m^{\lg 3} n^2)$.

La observación clave para mejorar exposec consiste en que $a^n = (a^{n/2})^2$ cuando n es par. Esto es interesante porque $a^{n/2}$ se puede calcular aproximadamente cuatro veces más deprisa que a^n con exposec , y basta una única elevación al cuadrado (que es una multiplicación) para obtener el resultado deseado a partir de $a^{n/2}$. Esto produce la recurrencia siguiente:

$$a^n = \begin{cases} a & \text{si } n = 1 \\ (a^{n/2})^2 & \text{si } n \text{ es par} \\ a \times a^{n-1} & \text{en caso contrario} \end{cases}$$

Por ejemplo,

$$a^{29} = a \ a^{28} = a(a^{14})^2 = a((a^7)^2)^2 = \dots = a((a(a \ a^2)^2)^2)^2$$

que sólo utiliza tres multiplicaciones y cuatro elevaciones al cuadrado en lugar de las 28 multiplicaciones que se necesitaban con *exposec*. La recurrencia anterior da lugar al siguiente algoritmo:

función *expoDV*(*a*, *n*)

si *n* = 1 **entonces devolver** *a*

si *n* es par **entonces devolver** [*expoDV*(*a*, *n*/2)]²

devolver *a* × *expoDV*(*a*, *n* − 1)

Para analizar la eficiencia de este algoritmo, nos concentraremos primero en el número de multiplicaciones (contando la elevación al cuadrado como una multiplicación) efectuadas por una llamada a *expoDV*(*a*, *n*). Observemos que el flujo de control del algoritmo no depende del valor de *a*, y por tanto el número de multiplicaciones sólo es una función del exponente *n*; lo denotaremos *N*(*n*).

Cuando *n* = 1 no se efectúa ninguna multiplicación, así que *N*(1) = 0. Cuando *n* es par, se efectúa una multiplicación (la elevación al cuadrado de *a^{n/2}*) además de las *N*(*n*/2) multiplicaciones implicadas en la llamada recursiva a *expoDV*(*a*, *n*/2). Cuando *n* es impar, se efectúa una multiplicación (la de *a* por *aⁿ⁻¹*) además de las *N*(*n* − 1) multiplicaciones requeridas por la llamada recursiva a *expoDV*(*a*, *n* − 1). Entonces tenemos la recurrencia siguiente.

$$N(n) = \begin{cases} 0 & \text{si } n = 1 \\ N(n/2) + 1 & \text{si } n \text{ es par} \\ N(n-1) + 1 & \text{en caso contrario} \end{cases} \quad (7.12)$$

Una característica poco habitual de esta recurrencia es que no da lugar a una función creciente de *n*. Por ejemplo:

$$\begin{aligned} N(15) &= N(14) + 1 = N(7) + 2 = N(6) + 3 = N(3) + 4 = N(2) + 5 \\ &= N(1) + 6 = 6 \end{aligned}$$

$$\text{mientras que } N(16) = N(8) + 1 = N(4) + 2 = N(2) + 3 = N(1) + 4 = 4.$$

Esta función no es ni siquiera *eventualmente* no decreciente; véase el Problema 7.26. Por tanto, no se puede utilizar directamente la ecuación 7.1.

Para tratar esta recurrencia, es útil acotar superior e inferiormente la función mediante funciones no decrecientes. Cuando $n > 1$ es impar:

$$N(n) = N(n-1) + 1 = N((n-1)/2) + 2 = N(\lfloor n/2 \rfloor) + 2$$

Por otra parte, cuando n es par, $N(n) = N(\lfloor n/2 \rfloor) + 1$, puesto que $\lfloor n/2 \rfloor = n/2$ en ese caso. Por tanto,

$$N(\lfloor n/2 \rfloor) + 1 \leq N(n) \leq N(\lfloor n/2 \rfloor) + 2 \quad (7.13)$$

para todo $n > 1$. Sean N_1 y N_2 dos funciones definidas mediante

$$N_i(n) = \begin{cases} 0 & \text{si } n = 1 \\ N_i(\lfloor n/2 \rfloor) + i & \text{en caso contrario} \end{cases} \quad (7.14)$$

para $i = 1$ e $i = 2$. Empleando la ecuación 7.13, es fácil comprobar por inducción matemática que $N_1(n) \leq N_2(n)$ para todo n ; véase el Problema 7.27. Pero ahora tanto N_1 como N_2 son no decrecientes (también es fácil demostrarlo por inducción matemática). Además, $\lfloor n/2 \rfloor = n/2$ cuando $n > 1$ es una potencia de 2. La ecuación 7.1 es aplicable a $N_1(n)$ y a $N_2(n)$ con $\ell = 1$, $b = 2$ y $k = 0$, para dar que estas dos funciones están en $\Theta(\log n)$. Concluimos que $N(n)$ se encuentra también en $\Theta(\log n)$. Recordemos que *exposec* requería un número de multiplicaciones que estaba en $\Theta(n)$ para efectuar la misma exponenciación. Por tanto, *expoDV* es más eficiente que su competidor empleando este criterio. Queda por ver si también es significativamente mejor cuando se tiene en cuenta el *tiempo* invertido en esas multiplicaciones.

Sea una vez más $M(q, s)$ el tiempo necesario para multiplicar dos enteros de tamaños q y s , y denotemos ahora mediante $T(m, n)$ el tiempo que invierte en multiplicar una llamada a *expoDV*(a, n), en donde m es el tamaño de a . Recordemos que el tamaño de a' es como mucho im . La inspección del algoritmo *expoDV* da lugar a la recurrencia siguiente:

$$T(m, n) \leq \begin{cases} 0 & \text{si } n = 1 \\ T(m, n/2) + M(mn/2, mn/2) & \text{si } n \text{ es par} \\ T(m, n-1) + M(m, (n-1)m) & \text{en caso contrario} \end{cases} \quad (7.15)$$

Como en la recurrencia para N , esto implica que

$$T(m, n) \leq T(m, \lfloor n/2 \rfloor) + M(m \lfloor n/2 \rfloor, m \lfloor n/2 \rfloor) + M(m, (n-1)m)$$

para todo $n > 1$. Si $M(q, s) \in \Theta(sq^{\alpha-1})$ para alguna constante α cuando $s \geq q$ ($\alpha = 2$ con el algoritmo clásico de multiplicación, mientras que $\alpha = \lg 3$ con multiplicación por divide y vencerás), el problema 7.29 muestra que $T(m, n) \in O(m^\alpha n^\alpha)$. La cota inferior equivalente es más fácil de obtener. La última multiplicación efectuada por $\text{expoDV}(a, n)$, o la penúltima, dependiendo de si n es par o impar, consiste en elevar al cuadrado $a^{\lfloor n/2 \rfloor}$, que es un entero de tamaño mayor que $(m-1)\lfloor n/2 \rfloor$. Esto cuesta como mínimo $M((m-1)\lfloor n/2 \rfloor, (m-1)\lfloor n/2 \rfloor)$, que se encuentra en $\Omega(((m-1)\lfloor n/2 \rfloor)^\alpha)$, y por tanto en $\Omega(m^\alpha n^\alpha)$. Concluimos que $T(m, n) \in \Theta(m^\alpha n^\alpha)$.

Para resumir, la tabla siguiente nos da el tiempo necesario para calcular a^n , en donde m es el tamaño de a , dependiendo de si utilizamos exposec o expoDV , y de si utilizamos el algoritmo de multiplicación clásico o el de divide y vencerás (D y V).

	multiplicación	
	clásica	D y V
exposec	$\Theta(m^2 n^2)$	$\Theta(m^{\lg 3} n^2)$
expoDV	$\Theta(m^2 n^2)$	$\Theta(m^{\lg 3} n^{\lg 3})$

Resulta interesante constatar que no ganamos nada —salvo quizá un factor constante de velocidad— al ser sólo medio despiertos: tanto exposec como expoDV requieren un tiempo que está en $\Theta(m^2 n^2)$ cuando se utiliza el algoritmo clásico de multiplicación. Esto es cierto aun cuando expoDV utiliza una cantidad de multiplicaciones exponencialmente menor que exposec .

Tal como sucede en la búsqueda binaria, el algoritmo expoDV requiere solamente una llamada recursiva en un caso de menor tamaño. Por tanto, es un ejemplo de simplificación, más que de divide y vencerás. Sin embargo, esta llamada recursiva no se encuentra dinámicamente al final del algoritmo, puesto que después de ella queda por hacer una multiplicación. Esto hace que sea más difícil encontrar una versión iterativa. Sin embargo, tal algoritmo existe y se corresponde intuitivamente con calcular $a^{2^9} = a^1 a^4 a^8 a^{16}$. Lo indicamos sin más dilación, observando que esta versión sencilla hace sistemáticamente dos multiplicaciones inútiles, incluyendo la última elevación al cuadrado de x .

```

función expoiter( $a, n$ )
   $i \leftarrow n; r \leftarrow 1; x \leftarrow a$ 
  mientras  $i > 0$  hacer
    si  $i$  es impar entonces  $r \leftarrow rx$ 
     $x \leftarrow x^2$ 
     $i \leftarrow i \div 2$ 
  devolver  $r$ 
```


7.8 ENSAMBLANDO TODAS LAS PIEZAS: INTRODUCCIÓN A LA CRIPTOGRAFÍA

Retrotrayéndonos a la sección anterior, resulta descorazonador que una reducción exponencial del número de multiplicaciones necesario para calcular a^n no se traduzca en un espectacular ahorro de tiempo de ejecución. Sin embargo, existen aplicaciones para las cuales *es* razonable contar todas las multiplicaciones como de un mismo coste. Tal es el caso si estamos interesados en la aritmética *modular*, esto es, en el cálculo de a^n módulo algún tercer entero z . Recuerde que $x \bmod z$ denota el resto de la división entera de x por z . Por ejemplo, $25 \bmod 7 = 4$ porque $25 = 3 \times 7 + 4$. Si x e y son dos enteros entre 0 y $z-1$, y si z es un entero de tamaño m , entonces la multiplicación modular $xy \bmod z$ necesita una multiplicación entera ordinaria de dos enteros de tamaño m como máximo, dando lugar a un entero de tamaño $2m$ como máximo, seguida por división del producto por z , un entero de tamaño m , para calcular el resto de la división. Por tanto, el tiempo que requiere cada multiplicación modular es más bien independiente de los dos números involucrados. Utilizaremos dos propiedades elementales de la aritmética modular; véase el Problema 7.30.

$$xy \bmod z = [(x \bmod z) \times (y \bmod z)] \bmod z$$

y

$$(x \bmod z)^y \bmod z = x^y \bmod z$$

Por tanto, *exposec*, *expoDV* y *expoiter* se pueden adaptar para calcular $a^n \bmod z$ en aritmética modular, sin tener que manipular nunca enteros que sean más grandes que $\max(a, z^2)$. Para ello, basta reducir módulo z después de cada multiplicación. Por ejemplo, *expoiter* da lugar al algoritmo siguiente:

```

función expomod( $a, n, z$ )
  {Calcula  $a^n \bmod z$ }
  mientras  $i > 0$  hacer
    si  $i$  es impar entonces  $r \leftarrow a \bmod z$ 
     $x \leftarrow x^2 \bmod z$ 
     $i \leftarrow i \div 2$ 
  devolver  $r$ 

```

El análisis de la sección anterior se aplica *mutatis mutandis* para concluir que este algoritmo requiere solamente un número de multiplicaciones modulares que está en $\Theta(\log n)$ para calcular $a^n \bmod z$. Un análisis más preciso muestra que el número de multiplicaciones modulares es igual al número de *bits* que hay en la expansión binaria de n , más el número de estos *bits* que son iguales a 1; por tanto, es aproximadamente igual a $\frac{3}{2} \lg n$ para un n típico. Por contraste, el algoritmo correspondiente a *exposec* requiere $n - 1$ multiplicaciones de éstas para todo n . Para

concretar, supongamos que se desea calcular $a^n \bmod z$, en donde a , n y z son números de 200 dígitos, y supongamos también que los números de este tamaño se pueden multiplicar módulo z en un milisegundo. Nuestro algoritmo *expomod* calculará $a^n \bmod z$ en menos de un segundo. El algoritmo correspondiente a *exposec* requeriría aproximadamente ¡179 veces la edad del Universo para la misma tarea!

Con todo lo impresionante que resulta esto, quizá se pregunte quién podría tener la necesidad de calcular exponenciaciones modulares tan enormes en la vida real. Resulta que la *criptografía* moderna, que es el arte y la ciencia de la comunicación secreta a través de canales poco seguros, depende crucialmente de ella. Consideremos dos partes que llamaremos Alicia y Benito, y supongamos que Alicia desea enviar un mensaje secreto m a Benito, a través de un canal que podría estar sometido a escuchas. Para evitar que otras personas lean el mensaje, Alicia lo transforma en un *texto cifrado* c , que envía a Benito. Esta transformación es el resultado de un algoritmo de *cifrado* cuya salida depende no solamente del mensaje m sino también de otro parámetro k conocido con el nombre de *clave*. Clásicamente, esta clave es información secreta, que debe haber sido acordada entre Alicia y Benito antes de que pueda tener lugar una conversación secreta. A partir de c y de su conocimiento de k , Benito puede reconstruir el verdadero mensaje m de Alicia. Estos sistemas de criptografía confían en que el escucha que intercepte c pero no conozca k no sea capaz de determinar m a partir de la información disponible.

Esta aproximación a la criptografía ha sido utilizada con más o menos éxito a lo largo de la Historia. Su requisito es que las partes deban compartir información secreta antes que la comunicación pueda resultar aceptable para militares y diplomáticos, pero no para el ciudadano ordinario. En la era de las superautopistas electrónicas, es deseable para dos ciudadanos cualesquiera poder comunicarse en privado sin coordinación previa. ¿Pueden comunicarse Alicia y Benito secretamente ante la vista de una tercera parte, si no comparten un secreto antes de establecer la comunicación? La era de la *criptografía de clave pública* nació cuando la idea de que esto pudiera ser posible fue desarrollada por Diffie, Hellman y Merkle a mediados de los años setenta. Aquí presentamos la sorprendentemente sencilla solución que descubrieron pocos años después Rivest, Shamir y Adleman, y que recibió el nombre de *sistema criptográfico RSA* a partir de los nombres de sus inventores.

Consideremos dos números primos p y q de 100 dígitos, seleccionados aleatoriamente por Benito; véase la Sección 10.6.2 para un algoritmo eficiente capaz de comprobar la primalidad de números tan grandes. Sea z el producto de p y q . Benito puede calcular eficientemente z a partir de p y q . Sin embargo, ningún algoritmo conocido puede recalcular p y q a partir de z dentro del tiempo de duración del Universo, ni siquiera empleando la computadora más rápida conocida en el momento de escribir esto. Sea ϕ igual a $(p-1)(q-1)$. Sea n un entero seleccionado aleatoriamente por Benito entre 1 y $z-1$, que no tenga factores comunes con ϕ . (No es necesario que Benito compruebe explícitamente el que n tenga la propiedad deseada, porque lo averiguará en seguida de no ser así.) La teoría elemental de números nos dice que existe un único entero s entre 1 y $z-1$ tal que $ns \bmod$

$\phi = 1$. Además, s es fácil de calcular a partir de n y ϕ —véase el Problema 7.31— y su existencia es la demostración de que n y ϕ no poseen factores comunes. Si s no existe, Benito tiene que seleccionar aleatoriamente un nuevo valor para n ; todos los intentos tienen buenas probabilidades de éxito. El teorema clave es que $a^n \bmod z = a$ siempre que $0 \leq a < z$ y $x \bmod \phi = 1$.

Para permitir que Alicia o alguien más se comuniquen privadamente con él, Benito hace pública su selección de z y n , pero mantiene secreto el valor de s . Sea m un mensaje que Alicia desea transmitir a Benito. Empleando una codificación estándar como ASCII, Alicia transforma su mensaje en una sucesión de *bits*, que interpreta como un número a . Supongamos por sencillez que $0 \leq a \leq z - 1$; en caso contrario, Alicia puede cortar su mensaje m en trozos de tamaño adecuado. A continuación, Alicia utiliza el algoritmo *expomod* para calcular $c = a^n \bmod z$, que envía a Benito a través de un canal sin protección. Empleando su conocimiento privado de s , Benito calcula a , y por tanto el mensaje m de Alicia, mediante una llamada a *expomod*(c, s, z). Esto funciona porque

$$c^s \bmod z = (a^n \bmod z)^s \bmod z = (a^n)^s \bmod z = a^{ns} \bmod z = a.$$

Consideremos ahora la tarea del escucha. Suponiendo que haya interceptado todas las comunicaciones entre Alicia y Benito, el escucha conoce z , n y c . Su objetivo es determinar el mensaje a de Alicia, que es el único número entre 0 y $z - 1$ tal que $c = a^n \bmod z$. Entonces tiene que calcular la raíz n -ésima de c módulo z . Para este cálculo no se conoce un algoritmo eficiente: las exponenciaciones modulares se pueden calcular eficientemente con *expomod*, pero parece que el proceso inverso no es factible. El mejor método que se conoce en la actualidad es el evidente: factorizar z en p y q , calcular ϕ como $(p-1)(q-1)$, utilizar el problema 7.31 para calcular s a partir de n y ϕ , y calcular $a = c^s \bmod z$ exactamente igual que lo habría hecho Benito. Todos los pasos de este ataque son factibles, menos el primero: la factorización de un número de 200 dígitos está fuera del alcance de la tecnología actual. Por tanto, la ventaja de Benito para descifrar mensajes destinados a él surge del hecho de que solamente él conoce los factores de z , que son necesarios para calcular ϕ y s . Este conocimiento no proviene de sus capacidades de factorización, sino más bien del hecho de que ha seleccionado primero los factores de z , y ha calculado z a partir de ellos.

En el momento de escribir esto, la seguridad de este esquema criptográfico no se ha establecido matemáticamente: la factorización puede resultar ser fácil, o puede incluso ser innecesaria, para violar este sistema. Además, se *conoce* un algoritmo de factorización eficiente, pero requiere la existencia de una *computadora cuántica*, dispositivo cuya construcción va más allá de los límites de la tecnología actual; véase la Sección 12.6. Sin embargo, el sistema secreto que acabamos de describir goza de amplia aceptación como una de las mejores invenciones en la historia de la criptografía.